

BÁO CÁO ĐỒ ÁN / MÔN THỰC TẬP CƠ SỞ
TÌM HIỂU VÀ ỨNG DỤNG MỘT SỐ CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN
TRONG PHÁT TRIỂN GAME 2D

Sinh viên thực hiện : NGUYỄN QUANG TRUNG

Mã số sinh viên : B23DCKH122

Lớp : B23CQKH02-B

Năm học 2026

MỤC LỤC

NHẬN XÉT CỦA GIẢNG VIÊN

TÓM TẮT

DANH MỤC TỪ VIẾT TẮT

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề

1.2 Mục tiêu đề tài

1.3 Phạm vi và giới hạn

1.4 Công nghệ sử dụng

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan kiến trúc phần mềm trong game

2.1.1 Kiến trúc OOP truyền thống

2.1.2 Data-Oriented Design

2.2 Entity-Component-System

2.2.1 Khái niệm Entity, Component, System

2.2.2 Luồng xử lý dữ liệu trong ECS

2.2.3 Lợi ích về bảo trì và mở rộng

2.3 Cấu trúc dữ liệu Sparse Set

2.3.1 Nguyên lý hoạt động

2.3.2 Độ phức tạp thời gian và không gian

2.3.3 Ưu điểm với CPU cache so với `std::unordered_map`

2.4 Spatial Partitioning

2.4.1 Quadtree

2.4.2 Uniform Grid

2.4.3 So sánh và lựa chọn cấu trúc phù hợp

2.5 Finite State Machine

2.5.1 Định nghĩa và mô hình

2.5.2 Ứng dụng trong AI nhân vật game

2.5.3 Hướng mở rộng : HFSM, Behavior Tree

2.6 Kỹ thuật mạng trong game real-time

2.6.1 Mô hình Client-Server

2.6.2 Client-side Prediction

2.6.3 Input Buffering

2.6.4 Reconciliation và Rollback

CHƯƠNG 3 : THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng thể

3.2 ECS Core

3.2.1 EntityManager - cấp phát và tái sử dụng Entity ID

3.2.2 Memory Pool với SparseSet

3.2.3 SystemManager

- 3.3 Thiết kế Sparse Set
 - 3.3.1 Cấu trúc mảng thành phần
 - 3.3.2 Thiết kế các thuật toán cốt lõi
- 3.4 Spatial Partitioning System
 - 3.4.1 Lựa chọn cấu trúc (Uniform Grid)
 - 3.4.2 Tích hợp vào Collision System trong ECS
- 3.5 AI System - Finite State Machine
 - 3.5.1 Thiết kế State Machine
 - 3.5.2 Chuyển dịch giữa các trạng thái
 - 3.5.3 Tích hợp FSM vào ECS
- 3.6 Network Layer
 - 3.6.1 Kiến trúc Server - Client giả lập
 - 3.6.2 Input Buffering
 - 3.6.3 Client-side Prediction
 - 3.6.4 Reconciliation khi server state lệch client
- 3.7 Profiler

CHƯƠNG 4 : TRIỂN KHAI VÀ CÀI ĐẶT

- 4.1 Môi trường phát triển
- 4.2 Cài đặt ECS Core
 - 4.2.1 EntityManager
 - 4.2.2 ComponentStorage
 - 4.2.3 SystemManager
- 4.3 Cài đặt Spatial Partitioning
 - 4.3.1 Uniform Grid
 - 4.3.2 Collision Detection System
- 4.4 Cài đặt Finite State Machine
 - 4.4.1 StateMachine template
 - 4.4.2 Định nghĩa các State dựa trên tập hợp Component
 - 4.4.3 Kết nối FSM với ECS Component
- 4.5 Cài đặt Network Layer
 - 4.5.1 Giả lập độ trễ mạng
 - 4.5.2 Input Buffering
 - 4.5.3 Client-side Prediction & Reconciliation
- 4.6 Cài đặt Profiler

CHƯƠNG 5 : THỰC NGHIỆM VÀ ĐÁNH GIÁ

- 5.1 Đánh giá hiệu năng kiến trúc ECS
 - 5.1.1 Profiling : OOP (std::vector<GameObject*> vs ECS (SoA))
 - 5.1.2 Kết quả : bảng + biểu đồ so sánh
 - 5.1.3 Phân tích và nhận xét
- 5.2 Đánh giá Sparse Set

5.2.1 So sánh tốc độ add/remove/iterate vs std::unordered_map

5.2.2 Kết quả : bảng + biểu đồ so sánh

5.3 Đánh giá FSM

5.3.1 Ưu điểm

5.3.2 Nhược điểm khi số trạng thái nhảy vọt

5.4 Đánh giá kỹ thuật mạng

5.4.1 Độ trễ cảm nhận : có với không có Client-side Prediction

5.4.2 Ưu điểm và nhược điểm

CHƯƠNG 6 : KẾT LUẬN

6.1 Kết quả đạt được

6.2 Hạn chế

6.3 Hướng phát triển

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Hà nội , ngày tháng năm
Giảng viên hướng dẫn

TÓM TẮT

Đề tài tập trung nghiên cứu và ứng dụng các cấu trúc dữ liệu và thuật toán nền tảng trong phát triển game 2D, sử dụng kiến trúc Entity-Component-System làm trục tổ chức hệ thống. Thay vì chú trọng gameplay, trọng tâm là cách tổ chức, lưu trữ và xử lý dữ liệu theo thời gian thực, bao gồm: SparseSet cho quản lý component, Spatial Partitioning cho phát hiện va chạm, Finite State Machine cho hành vi AI và các kỹ thuật mạng (Client-side Prediction, Input Buffering). Hệ thống được triển khai bằng C++ với SFML và ImGui hỗ trợ profiling trực quan

DANH MỤC TỪ VIẾT TẮT

ECS	Entity Component System
FSM	Finite State Machine
SFML	Simple and Fast Multimedia Library
CPU	Central Processing Unit
FPS	Frames Per Second
AI	Artificial Intelligence
RTT	Round-Trip Time
OOP	Object-Oriented Programming
DOD	Data-Oriented Design
AABB	Axis-Aligned Bounding Box
SoA	Structure of Arrays
AoS	Array of Structures
HFSM	Hierarchical Finite State Machine
RAII	Resource Acquisition Is Initialization
GUI	Graphical User Interface

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề

Trong phát triển game thời gian thực, việc duy trì tốc độ khung hình ổn định (tối thiểu 60 FPS) là yếu tố quyết định đến trải nghiệm người chơi. Với các tựa game 2D có số lượng thực thể lớn như hệ thống hạt, bão đạn hay hàng ngàn AI di chuyển cùng lúc, áp lực tính toán đè nặng lên CPU là rất lớn.

Mô hình lập trình hướng đối tượng truyền thống quản lý game qua cấu trúc cây và danh sách con trỏ `std::vector<GameObject*>` bộc lộ những hạn chế lớn về mặt hiệu năng:

1. Phân mảnh bộ nhớ: Dữ liệu bị rải rác trên bộ nhớ Heap, gây ra tỷ lệ trượt bộ nhớ đệm cao khi CPU duyệt qua các đối tượng để cập nhật logic.
2. Kế thừa công kênh: Các đối tượng phải mang theo nhiều thuộc tính dư thừa từ lớp cha và chịu chi phí gọi hàm ảo qua bảng vtable.

Để giải quyết bài toán nghẽn cổ chai này, xu hướng thiết kế hướng dữ liệu ra đời với đại diện là kiến trúc Entity - Component - System. Bằng cách tách biệt Định danh, Dữ liệu thuần và Logic, dữ liệu được sắp xếp liên tục trên bộ nhớ phẳng, giúp tối ưu hóa tối đa bộ nhớ đệm CPU Cache.

Tuy nhiên, việc tự xây dựng một Game Engine theo kiến trúc ECS từ con số không đòi hỏi phải giải quyết nhiều bài toán phức tạp về cấu trúc dữ liệu và giải thuật:

1. Quản lý và truy xuất component linh hoạt ở độ phức tạp $O(1)$ mà vẫn đảm bảo bộ nhớ liên tục thông qua cấu trúc Sparse Set.
2. Tối ưu hóa bài toán va chạm từ $O(N^2)$ xuống $O(kN)$ bằng các kỹ thuật phân vùng không gian như Uniform Grid hay Quadtree.
3. Tích hợp tầng mạng thời gian thực với các kỹ thuật Client-side Prediction, Input Buffering và Reconciliation dựa trên cấu trúc lưu trữ phẳng của ECS.

Từ những nhu cầu thực tế về tối ưu phần cứng và xử lý mạng, đề tài "Tìm hiểu và ứng dụng cấu trúc dữ liệu và thuật toán trong phát triển game 2D" được lựa chọn thực hiện nhằm tự nghiên cứu, cài đặt và đánh giá một giải pháp công nghệ game có hiệu năng cao bằng ngôn ngữ C++

1.2 Mục tiêu đề tài

Để giải quyết các vấn đề đã đặt ra, đề tài tập trung hoàn thành các mục tiêu cụ thể sau:

1. Xây dựng thành công lõi Engine: Tự cài đặt bằng C++ một hệ thống core ECS hoàn chỉnh gồm EntityManager, ComponentManager và SystemManager. Đảm

bảo hệ thống quản lý, cấp phát ID thực thể và lưu trữ component trên bộ nhớ phẳng một cách độc lập, không bị phụ thuộc vào các thư viện framework có sẵn.

2. Tối ưu hóa tốc độ truy xuất bằng Sparse Set: Áp dụng cấu trúc dữ liệu Sparse Set vào các Memory Pool lưu trữ component. Mục tiêu là đạt độ phức tạp $O(1)$ cho các thao tác thêm, xóa, kiểm tra linh hoạt mà vẫn giữ dữ liệu nằm liên tục nhằm giảm thiểu tối đa hiện tượng Cache Miss trên CPU.
3. Giải quyết bài toán va chạm và hiển thị: Triển khai cấu trúc phân vùng không gian Uniform Grid nhằm giảm độ phức tạp tính toán va chạm từ $O(N^2)$ xuống mức nguồn tối ưu hơn
4. Xử lý AI và Đồng bộ mạng giả lập: Xây dựng một Finite State Machine điều khiển hành vi AI của quái vật tích hợp mượt mà vào ECS. Cài đặt thành công một tầng mạng giả lập có độ trễ, áp dụng các kỹ thuật Client-side Prediction, Input Buffering và Reconciliation để xử lý hiện tượng giật lag, đảm bảo game chạy mượt ở phía người chơi.
5. Thực nghiệm, đo đạc và đánh giá: Tích hợp bộ công cụ Profiler kết hợp giao diện ImGui để đo đạc chính xác thời gian chạy của từng System. Lập bảng biểu, đồ thị so sánh trực quan hiệu năng giữa kiến trúc ECS (DOD) vừa làm với kiến trúc OOP truyền thống để chứng minh tính hiệu quả của đề tài.

1.3 Phạm vi và giới hạn

Do giới hạn về thời gian thực hiện và mục tiêu chính của môn học là tập trung vào cấu trúc dữ liệu và giải thuật tối ưu hiệu năng, đề tài được giới hạn trong phạm vi sau:

1. Về mặt đồ họa và Gameplay: Đề tài chỉ tập trung xây dựng môi trường game 2D mô phỏng, không đầu tư quá sâu vào việc xây dựng cốt truyện, lối chơi phức tạp hay thiết kế đồ họa chất lượng cao. Hình ảnh trong game chủ yếu sử dụng các khối hình học cơ bản để minh họa cho các hệ thống hoạt động.
2. Về môi trường mạng: Hệ thống không triển khai trên đường truyền mạng Internet thực tế hay xây dựng một Server vật lý độc lập. Thay vào đó, tầng mạng sẽ được xây dựng dưới dạng giả lập trên cùng một chương trình máy tính
3. Về mặt phần cứng và luồng xử lý: Trong khuôn khổ đề án này, lõi ECS Core và các System được thiết kế để chạy và tính toán trên một luồng chính duy nhất (Single-threading). Các kỹ thuật xử lý đa luồng hay lập trình song song chỉ dừng lại ở mức độ nghiên cứu lý thuyết và đề xuất trong hướng phát triển tương lai.
4. Về công cụ phát triển: Đề tài tự hiện thực các cấu trúc dữ liệu cốt lõi (như Sparse Set, Grid, State Machine) từ con số không bằng ngôn ngữ C++. Chỉ sử dụng thư viện SFML để hỗ trợ các tính năng cơ bản như mở cửa sổ, vẽ sprite 2D, bắt sự kiện bàn phím/chuột và thư viện ImGui để làm thành công cụ theo dõi (Profiler) hiệu năng trực quan.

1.4 Công nghệ sử dụng

Để hiện thực hóa các mục tiêu thiết kế hướng dữ liệu và tối ưu hóa hiệu năng, đề tài lựa chọn bộ công cụ và công nghệ phát triển tối giản, đi sâu vào kiểm soát phần cứng bao gồm:

1. Ngôn ngữ lập trình C++: Lựa chọn bắt buộc để xây dựng hệ thống core ECS từ con số không. C++ cung cấp khả năng quản lý bộ nhớ thủ công tuyệt đối, cho phép can thiệp sâu vào cách sắp xếp dữ liệu trên bộ nhớ RAM (Contiguous Memory Allocation), từ đó kiểm soát và tối ưu hóa tối đa cơ chế hoạt động của bộ nhớ đệm CPU Cache.
2. Thư viện đồ họa SFML (Simple and Fast Multimedia Library): Được sử dụng làm tầng giao tiếp phần cứng cơ bản cho game 2D. Thư viện này chỉ đảm nhận các nhiệm vụ: quản lý cửa sổ ứng dụng (Windowing), bắt sự kiện từ thiết bị ngoại vi (Keyboard/Mouse), xử lý âm thanh và render các hình ảnh, Sprite 2D lên màn hình thông qua OpenGL. SFML không can thiệp hay áp đặt bất kỳ cấu trúc quản lý đối tượng nào lên logic của Engine.
3. Thư viện ImGui (Dear ImGui): Thư viện giao tiếp người dùng đồ họa (GUI) dạng Immediate Mode, có trọng lượng cực nhẹ và tốc độ xử lý nhanh. ImGui được tích hợp trực tiếp vào vòng lặp game để xây dựng công cụ Profiler, cho phép hiển thị các bảng biểu, biểu đồ xung nhịp hệ thống và thông số FPS theo thời gian thực nhằm mục đích debug và đánh giá hiệu năng.
4. Trình biên dịch và Công cụ Build (Clang / Ninja / CMake): Sử dụng hệ thống CMake để quản lý mã nguồn linh hoạt, kết hợp trình biên dịch Clang và bộ công cụ build tăng tốc Ninja nhằm tối ưu hóa quá trình biên dịch mã nguồn C++, hỗ trợ phát hiện lỗi bộ nhớ sớm trong quá trình phát triển.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan kiến trúc phần mềm trong game

Kiến trúc phần mềm trong phát triển game có sự khác biệt lớn so với các ứng dụng doanh nghiệp thông thường. Thay vì xử lý dữ liệu theo dạng phản hồi yêu cầu, một game engine phải vận hành dựa trên một vòng lặp thời gian thực. Vòng lặp này liên tục thực hiện ba bước: Nhận tương tác, Cập nhật logic, và Vẽ lên màn hình với tần suất từ 60 đến hàng trăm lần mỗi giây. Do đó, cách tổ chức mã nguồn và cấu trúc dữ liệu ảnh hưởng trực tiếp đến tốc độ xử lý của toàn bộ hệ thống.

2.1.1 Kiến trúc OOP truyền thống

Trong nhiều thập kỷ, lập trình hướng đối tượng là mô hình thống trị trong công nghệ game. Kiến trúc này tổ chức hệ thống dựa trên tư duy mô hình hóa thế giới thực: mọi thực thể trong game đều là một đối tượng.

Ở các engine OOP đời đầu, tính kế thừa sâu được lạm dụng để mở rộng tính năng. Ví dụ, một đối tượng Monster sẽ kế thừa từ lớp Character, lớp này lại kế thừa từ MovableEntity, và trên cùng là lớp GameObject. Mô hình này nhanh chóng bộc lộ điểm yếu khi game mở rộng: cấu trúc trở nên cứng nhắc, dễ dẫn đến khủng hoảng đa kế thừa hoặc buộc đối tượng phải mang theo các thuộc tính thừa từ lớp cha mà nó không bao giờ dùng tới.

Để khắc phục, các engine hiện đại hơn chuyển sang kiến trúc GameObject - Component. Thay vì kế thừa, một GameObject đóng vai trò là một thùng chứa, sở hữu một danh sách các con trỏ trỏ đến các thành phần chức năng độc lập (như TransformComponent, RenderComponent, ColliderComponent). Logic của đối tượng sẽ được xử lý bằng cách duyệt qua danh sách thành phần này và gọi hàm cập nhật của từng cái.

Mặc dù giải quyết được bài toán linh hoạt trong thiết kế, mô hình OOP (bao gồm cả dạng Component hướng đối tượng) lại gặp phải một "bức tường" về hiệu năng phần cứng khi số lượng đối tượng tăng lên hàng ngàn, được gọi là nghẽn cổ chai do phân mảnh bộ nhớ:

1. Bộ nhớ rải rác (Memory Fragmentation): Trong OOP, các GameObject và các Component của chúng thường được cấp phát động (qua từ khóa new trong C++) tại các thời điểm khác nhau. Điều này khiến chúng nằm rải rác ở khắp nơi trên bộ nhớ Heap.
2. Tỷ lệ Cache Miss cao: Khi vòng lặp game duyệt qua danh sách các đối tượng để cập nhật logic (ví dụ: cập nhật vị trí), CPU sẽ cố gắng nạp dữ liệu từ RAM vào bộ nhớ đệm. Vì dữ liệu nằm rải rác không liên tục, CPU liên tục gặp hiện

tượng trượt bộ nhớ đệm. Mỗi lần Cache Miss, CPU phải dừng chu kỳ xử lý để chờ nạp dữ liệu từ RAM, gây lãng phí xung nhịp nghiêm trọng.

2.1.2 Data-Oriented Design

Thiết kế hướng dữ liệu (Data-Oriented Design - DOD) ra đời nhằm thay đổi hoàn toàn góc nhìn của lập trình viên: thay vì tập trung vào "Đối tượng và hành vi", DOD tập trung vào "Định dạng của dữ liệu và cách nó di chuyển qua phần cứng". DOD thừa nhận một thực tế của phần cứng hiện đại: Tốc độ xử lý của CPU đã vượt xa tốc độ đọc/ghi của bộ nhớ RAM. Do đó, rào cản lớn nhất của hiệu năng không nằm ở số lượng thuật toán, mà nằm ở cách sắp xếp dữ liệu trên bộ nhớ.

Triết lý cốt lõi của DOD bao gồm hai nguyên tắc chính:

1. Dữ liệu nằm liên tục: Thay vì lưu trữ một đối tượng chứa nhiều loại dữ liệu khác nhau, DOD khuyến khích tách rời các loại dữ liệu và xếp các dữ liệu cùng loại nằm sát nhau trong các mảng phẳng liên tiếp (Mô hình SoA - Structure of Arrays thay vì AoS - Array of Structures của OOP).
2. Tối ưu hóa Cache Locality: Khi các dữ liệu cùng loại (ví dụ: tất cả các tọa độ vị trí của quái vật) được xếp liên tiếp nhau trong bộ nhớ, một thao tác nạp dữ liệu của CPU sẽ mang theo một hàng dài các dữ liệu kế tiếp vào bộ nhớ đệm L1/L2 Cache cùng một lúc. Khi hệ thống duyệt đến phần tử tiếp theo, dữ liệu đã nằm sẵn trong Cache, giúp CPU xử lý logic với tốc độ tối đa mà không bị trễ.

DOD không cố gắng mô hình hóa thế giới game thành các thực thể thông minh tự quản lý logic. Ngược lại, nó nhìn game như một nhà máy biến đổi dữ liệu: Đầu vào là một luồng dữ liệu thô, đi qua các bộ lọc xử lý tập trung, và cho ra đầu ra là dữ liệu đã được cập nhật. Sự dịch chuyển tư duy này chính là tiền đề cho sự ra đời của kiến trúc ECS Core hiệu năng cao.

2.2 Entity-Component-System

Kiến trúc ECS là sự hiện thực hóa triết lý của tư duy Thiết kế hướng dữ liệu trong lập trình game. Thay vì gộp chung dữ liệu và logic vào một đối tượng như OOP, ECS phân rã trò chơi thành ba thành phần độc lập hoàn toàn.

2.2.1 Khái niệm Entity, Component, System

1. Entity: Định danh duy nhất của một đối tượng trong trò chơi. Về mặt mã nguồn, một Entity không phải là một Object công kênh, mà chỉ đơn thuần là một số nguyên nguyên thủy (uint32_t hoặc uint64_t). Bản thân Entity không chứa bất kỳ dữ liệu thuộc tính nào và cũng không có code logic; nó chỉ đóng vai trò như một chiếc "nhãn" hoặc mã khóa định danh để liên kết các Component lại với nhau.

2. **Component:** Nơi lưu trữ dữ liệu thuần túy. Mỗi Component là một cấu trúc dữ liệu thô (Struct hoặc Class chỉ có thuộc tính, không có hàm xử lý logic). Ví dụ: TransformComponent chỉ chứa tọa độ (x, y), VelocityComponent chỉ chứa vận tốc (vx, vy), RenderComponent chỉ chứa ID của texture. Các Component cùng loại của các Entity khác nhau sẽ được lưu trữ tập trung và liên tiếp nhau trong các mảng phẳng trên bộ nhớ.
3. **System (Hệ thống):** Nơi tập trung toàn bộ mã nguồn xử lý logic. Các System hoàn toàn không có trạng thái dữ liệu riêng. Mỗi System chỉ quan tâm đến một nhóm các Entity sở hữu tập hợp các Component nhất định. Ví dụ: MovementSystem sẽ lọc ra tất cả các Entity nào có cả hai thành phần TransformComponent và VelocityComponent để tính toán cộng vận tốc vào vị trí, ngoài ra nó bỏ qua tất cả các thực thể khác.

2.2.2 Luồng xử lý dữ liệu trong ECS

Trong mô hình ECS, luồng dữ liệu vận hành theo dạng tuyến tính và tập trung trong vòng lặp game (Game Loop). Thay vì từng đối tượng tự gọi hàm update() của riêng mình một cách cô lập, các System sẽ lần lượt làm việc theo thứ tự quy định.

1. Giai đoạn Triển khai Tuần tự (Compile-time Pipeline Execution)

- **Cơ chế vận hành:** Thông qua hàm `std::apply` kết hợp với biểu thức gập (Fold Expression), SystemManager sẽ duyệt qua từng System được lưu trữ trong `std::tuple` theo thứ tự từ trái sang phải một cách tuần tự (ví dụ: MovementSystem rồi đến CollisionSystem).
- **Áp dụng vào kiến trúc:** Mỗi System khi đến lượt sẽ tự động thực thi hàm `update()` riêng của mình. Do tất cả System đều được chia sẻ chung một con trỏ ngữ cảnh (scene), chúng có thể truy cập trực tiếp vào các bộ quản lý thành phần (Component Pools/Sparse Sets) nằm trong Scene đó.

2. Giai đoạn Lọc dữ liệu và Xử lý hàng loạt (Filtering & Batch Processing)

- **Cơ chế vận hành:** Bên trong hàm `update()` của từng System cụ thể, System đó sẽ thực hiện truy vấn các Entity sở hữu tập hợp Component mà nó quan tâm.
- **Tối ưu bộ nhớ Cache:** Khi đã xác định được các Entity hợp lệ, System thực hiện một vòng lặp phẳng duyệt tuyến tính qua các mảng dữ liệu Component liên tiếp trong bộ nhớ. Vì dữ liệu thành phần được tổ chức dưới dạng mảng đặc contiguous, CPU có thể kích hoạt cơ chế nạp trước dữ liệu, giảm thiểu tối đa hiện tượng Cache Miss và tối ưu hóa hiệu năng xử lý tính toán thô.

3. Giai đoạn Chuyển giao trạng thái liên tục (Continuous State Transition)

- Cơ chế vận hành: Do các System chạy tuần tự hoàn toàn trong một luồng (Single-thread), các thay đổi về dữ liệu do System đứng trước thực hiện (ví dụ: MovementSystem thay đổi vị trí trong TransformComponent) sẽ ngay lập tức được ghi thẳng vào vùng nhớ chung.
- Luồng dữ liệu kế tiếp: System đứng ngay sau nó trong std::tuple (ví dụ: RenderSystem) khi được gọi ở bước std::apply tiếp theo sẽ lập tức nhận vào dữ liệu mới nhất này để xử lý cho khung hình hiện tại. Quá trình này tạo thành một đường ống (pipeline) dẫn truyền dữ liệu khép kín, an toàn, không phát sinh chi phí sao chép bộ nhớ và đảm bảo tính nhất quán tuyệt đối trước khi render.

2.2.3 Lợi ích về bảo trì và mở rộng

Sự tách biệt hoàn toàn giữa dữ liệu và logic mang lại cho kiến trúc ECS những ưu điểm vượt trội về mặt kỹ thuật phần mềm:

1. Loại bỏ hoàn toàn khủng hoảng kế thừa: Thay vì phải đau đầu thiết kế cây kế thừa phức tạp, lập trình viên mở rộng tính năng của một thực thể bằng cơ chế kết hợp (Composition). Muốn một hòn đá bình thường biết bay? Chỉ cần gắn thêm VelocityComponent vào Entity hòn đá đó, ngay lập tức MovementSystem sẽ tự động nhận diện và xử lý nó mà không cần sửa một dòng code nào ở các lớp cha.
2. Tính module hóa cực cao (Decoupling): Các System hoạt động hoàn toàn độc lập với nhau. Bạn có thể dễ dàng thêm vào một ParticleSystem mới, sửa đổi AISystem, hoặc tắt bỏ tạm thời CollisionSystem để debug mà không sợ gây ra hiệu ứng cánh bướm làm đổ vỡ logic của các hệ thống khác.
3. Dễ dàng bảo trì và tái cấu trúc: Vì mã nguồn logic được gom về một nơi duy nhất (các System) thay vì rải rác ở hàng trăm file class đối tượng như OOP, việc tìm kiếm lỗi, tối ưu hóa thuật toán hoặc kiểm thử đơn vị (Unit Test) trở nên tập trung, tường minh và đơn giản hơn rất nhiều.

2.3 Cấu trúc dữ liệu Sparse Set

Để hiện thực hóa kiến trúc ECS với hiệu năng tối đa, hệ thống cần một cấu trúc dữ liệu cho phép quản lý và truy xuất các Component của Entity một cách linh hoạt.

Sparse Set chính là giải pháp cốt lõi đáp ứng hoàn hảo yêu cầu này: vừa đảm bảo tốc độ tra cứu cực nhanh, vừa giữ dữ liệu nằm liên tục trong bộ nhớ để tối ưu CPU Cache.

2.3.1 Nguyên lý hoạt động

Sparse Set cấu thành từ hai mảng số nguyên độc lập nhưng có mối quan hệ chặt chẽ với nhau, kết hợp cùng một mảng lưu trữ dữ liệu thô (Component Array):

1. Mảng Thừa (Sparse Array): Là một mảng có kích thước lớn, trong đó chỉ mục (index) của mảng chính là ID của Entity. Giá trị lưu tại ô Sparse[EntityID] sẽ

trở đến vị trí tương ứng của Entity đó trong Mảng Dày Đặc. Nếu ô đó chứa một giá trị đặc biệt (ví dụ: `max_int`), nghĩa là Entity đó không sở hữu Component này.

2. Mảng Dày Đặc (Dense Array): Là một mảng phẳng được xếp liền kề, không có khoảng trống. Mảng này lưu trữ trực tiếp danh sách các Entity ID đang sở hữu Component. Kích thước của mảng này tăng giảm linh hoạt dựa trên số lượng thực thể thực tế đang kích hoạt thành phần đó.
3. Mảng Dữ Liệu (Component Array): Là một mảng phẳng có kích thước và cơ chế quản lý chỉ mục song song 1:1 với Mảng Dày Đặc, dùng để lưu trữ dữ liệu thô của Component (ví dụ: `TransformComponent`).

Cơ chế liên kết kép (Tương quan chỉ mục): Khi một Entity sở hữu Component, mối quan hệ sau luôn được thỏa mãn: $\text{Dense}[\text{Sparse}[\text{EntityID}]] = \text{EntityID}$

Khi một Entity bị xóa bỏ Component, cấu trúc này áp dụng kỹ thuật "Swap and Pop": Hoán đổi phần tử bị xóa với phần tử cuối cùng trong Mảng Dày Đặc (và Mảng Dữ Liệu), sau đó cập nhật lại chỉ mục tương ứng trong Mảng Thừa và giảm kích thước mảng dày xuống 1 đơn vị. Kỹ thuật này giúp loại bỏ phần tử ở giữa mảng mà không cần phải dịch chuyển toàn bộ các phần tử phía sau.

2.3.2 Độ phức tạp thời gian và không gian

Nhờ vào nguyên lý ánh xạ chỉ mục trực tiếp, Sparse Set sở hữu hiệu năng cực kỳ ấn tượng về mặt thuật toán:

1. Độ phức tạp thời gian:
 - + Tra cứu : $O(1)$ - Chỉ cần truy cập trực tiếp qua chỉ mục `Sparse[EntityID]`.
 - + Thêm mới : $O(1)$ - Đẩy phần tử vào cuối Mảng Dày Đặc và cập nhật Mảng Thừa.
 - + Xóa bỏ : $O(1)$ - Áp dụng cơ chế Swap and Pop để đưa phần tử cần xóa về cuối mảng và giải phóng trong một chu kỳ tính toán.
 - + Duyệt tuần tự : $O(N)$ - Duyệt tuyến tính từ đầu đến cuối Mảng Dày Đặc/Mảng Dữ Liệu một cách mạch lạc.
2. Độ phức tạp không gian: $O(U + N)$

Trong đó U (Universe) là giá trị ID tối đa của Entity (độ dài tối đa của Mảng Thừa), và N là số lượng thực thể thực tế đang sở hữu Component (độ dài của Mảng Dày Đặc). Sparse Set đánh đổi một khoảng không gian trống trong Mảng Thừa để lấy tốc độ xử lý và sự liên tục của dữ liệu.

2.3.3 Ưu điểm với CPU Cache so với `std::unordered_map`

Trong thư viện chuẩn C++, `std::unordered_map` thường là lựa chọn đầu tiên khi lập trình viên nghĩ đến bài toán tìm kiếm $O(1)$. Tuy nhiên, trong môi trường Game Engine tần suất cao, `std::unordered_map` bộc lộ những nhược điểm chí mạng về mặt phần cứng khi so sánh với Sparse Set:

Tiêu chí	<code>std::unordered_map</code>	Sparse Set
Cấu trúc bộ nhớ	Sử dụng cơ chế băm và danh sách liên kết (Node-based). Các node dữ liệu bị cấp phát rải rác trên Heap.	Dữ liệu Component được gom cụm và xếp liên tiếp trong một mảng phẳng duy nhất.
Hành vi CPU Cache	Khi duyệt map, con trỏ nhảy liên tục giữa các vùng nhớ phân mảnh, gây ra tỷ lệ Cache Misses rất cao.	CPU nạp trước (Prefetch) dữ liệu liên tiếp vào L1/L2 Cache. Tỷ lệ Cache Hits đạt mức tối đa.
Chi phí tính toán	Tốn tài nguyên tính toán hàm băm (Hash Function) và xử lý xung đột băm (Collisions).	Chỉ sử dụng các phép toán truy cập mảng qua chỉ mục số nguyên thô, không tốn chi phí tính toán phụ.
Thao tác xóa	Việc giải phóng node đơn lẻ làm tăng hiện tượng phân mảnh bộ nhớ của hệ điều hành.	Thao tác Swap and Pop giúp giữ mảng luôn gọn gàng ở bộ nhớ phẳng mà không cần hủy/cấp phát lại vùng nhớ.

2.4 Spatial Partitioning

Trong các trò chơi 2D, việc kiểm tra va chạm giữa các đối tượng là một trong những tác vụ tiêu tốn nhiều tài nguyên CPU nhất. Nếu sử dụng thuật toán kiểm tra gốc (Brute-force), hệ thống phải lấy từng đối tượng so sánh với tất cả các đối tượng còn lại, dẫn đến độ phức tạp thuật toán là $O(N^2)$. Khi số lượng thực thể N đạt đến hàng ngàn, vòng lặp game sẽ lập tức bị nghẽn (bottleneck). Các kỹ thuật phân vùng không

gian ra đời nhằm chia nhỏ bản đồ thành các vùng quản lý cục bộ, giúp giới hạn phạm vi kiểm tra va chạm về mức tuyến tính hoặc tiệm cận tuyến tính.

2.4.1 Quadtree

Quadtree là một cấu trúc dữ liệu dạng cây, trong đó mỗi nút trong (Internal Node) có chính xác bốn nút con. Nguyên lý hoạt động của Quadtree dựa trên cơ chế phân chia không gian hai chiều theo dạng đệ quy:

1. Cơ chế phân tách: Ban đầu, toàn bộ không gian bản đồ game được quản lý bởi một nút gốc (Root Node). Khi số lượng thực thể nằm trong vùng không gian của một nút vượt quá một ngưỡng giới hạn định trước, nút đó sẽ tự động phân tách không gian của nó thành bốn vùng nhỏ hơn có diện tích bằng nhau (tương ứng với 4 hướng: Đông Bắc, Tây Bắc, Tây Nam, Đông Nam).
2. Phân phối phần tử: Các thực thể cũ và mới sẽ được đẩy xuống các nút con tương ứng với vị trí tọa độ của chúng. Quá trình này lặp lại đệ quy cho đến khi đạt độ sâu tối đa của cây hoặc số thực thể ở mỗi vùng nằm dưới ngưỡng giới hạn.
3. Truy vấn: Khi cần kiểm tra va chạm cho một đối tượng, hệ thống chỉ duyệt từ nút gốc xuống các nút lá chứa đối tượng đó, bỏ qua hoàn toàn các nhánh cây thuộc các vùng không gian xa xôi. Độ phức tạp tính toán trung bình được giảm xuống mức $O(N \log N)$.

2.4.2 Uniform Grid

Uniform Grid là kỹ thuật phân chia không gian trò chơi thành một mạng lưới gồm các ô vuông có kích thước cố định bằng nhau, giống như một bàn cờ vua.

1. Cơ chế lưu trữ: Kích thước của mỗi ô lưới thường được thiết lập lớn hơn hoặc bằng kích thước của thực thể lớn nhất trong game. Hệ thống quản lý lưới bằng một mảng hai chiều, hoặc sử dụng một mảng một chiều kết hợp công thức ánh xạ tọa độ thực (x, y) của đối tượng vào chỉ mục ô lưới.
2. Cập nhật dữ liệu: Ở mỗi khung hình, dựa vào tọa độ TransformComponent hiện tại trong hệ thống ECS, thực thể sẽ được đăng ký ID vào ô lưới tương ứng. Nếu thực thể di chuyển sang ô khác, nó sẽ xóa ID ở ô cũ và thêm vào ô mới.
3. Truy vấn: Khi một hệ thống (ví dụ: CollisionSystem) muốn tìm các đối tượng có khả năng va chạm với Entity A, nó chỉ cần truy cập trực tiếp vào ô lưới mà A đang đứng và 8 ô lưới xung quanh (Neighbor Cells). Việc truy cập này đạt độ phức tạp thời gian lý tưởng là $O(1)$.

2.4.3 So sánh và lựa chọn cấu trúc phù hợp

Mỗi cấu trúc dữ liệu phân vùng đều có những ưu điểm và nhược điểm riêng dựa trên kịch bản phân bố của các thực thể trong trò chơi:

Tiêu chí so sánh	Cây phân nhánh bốn	Cấu trúc lưới đều
Độ phức tạp truy xuất	Trung bình $O(\log N)$ do phải duyệt cây theo cấu trúc phân cấp đệ quy.	Lý tưởng $O(1)$ thông qua các phép toán chia số nguyên để tìm chỉ mục ô.
Phân bố không gian	Rất linh hoạt. Tự động thích ứng tốt với môi trường có mật độ thực thể phân bố không đều (co cụm).	Cố định. Hoạt động kém hiệu quả nếu bản đồ quá rộng nhưng thực thể chỉ tập trung vào một góc nhỏ.
Chi phí cập nhật	Cao. Khi các đối tượng di chuyển liên tục, việc tái cấu trúc cây (hủy, gộp, phân tách nút) tốn nhiều CPU.	Rất thấp. Thao tác cập nhật chỉ là tính lại chỉ mục và di chuyển ID giữa các ô mảng phẳng.
Tối ưu bộ nhớ đệm	Kém hơn do cấu trúc cây lưu dạng con trỏ Node phân mảnh trên bộ nhớ Heap.	Rất cao nếu lưu trữ danh sách Entity trong ô dưới dạng mảng phẳng liên tục (DOD).

2.5 Finite State Machine

Trong phát triển game, việc quản lý các hành vi phức tạp và có tính chu kỳ của nhân vật (như quái vật, NPC) đòi hỏi một mô hình điều khiển logic chặt chẽ. Mô hình trạng thái hữu hạn (FSM) là giải pháp kinh điển và hiệu quả nhất để giải quyết bài toán này.

2.5.1 Định nghĩa và mô hình

Về mặt lý thuyết toán học, Finite State Machine (FSM) là một mô hình tính toán dựa trên một ô-tô-mát hữu hạn trạng thái. Một hệ thống FSM tiêu chuẩn được định nghĩa bằng bộ 5 thành phần cốt lõi:

1. Tập hợp các trạng thái: Một danh sách hữu hạn các trạng thái mà hệ thống có thể chuyển đổi qua lại (ví dụ trong game: Idle, Patrol, Chase, Attack).
2. Tập hợp các sự kiện kích hoạt: Các điều kiện đầu vào từ môi trường game (ví dụ: nhìn thấy người chơi, mất dấu, lượng máu thấp).

3. Hàm chuyển dịch trạng thái: Logic toán học quy định việc chuyển từ trạng thái này sang trạng thái khác khi gặp sự kiện tương ứng.
4. Trạng thái khởi đầu : Trạng thái mặc định của thực thể khi vừa được tạo ra.
5. Tập hợp các trạng thái kết thúc : Trạng thái khi thực thể bị hủy bỏ hoàn toàn (ví dụ: Die).

Tại một thời điểm cụ thể, thực thể chỉ được phép nằm ở một và chỉ một trạng thái duy nhất (gọi là trạng thái hiện tại - Current State). Khi có sự kiện hợp lệ tác động, hệ thống sẽ thực thi hàm chuyển dịch để cập nhật trạng thái mới.

2.5.2 Ứng dụng trong AI nhân vật game

Trong một trò chơi 2D, FSM đóng vai trò là "bộ não" điều khiển hành vi của quái vật hoặc NPC một cách tự động thông qua việc phân chia logic thành các giai đoạn vòng đời của trạng thái:

1. Hành vi bên trong trạng thái : Mỗi trạng thái cụ thể thường quản lý ba hàm chức năng độc lập:
 - + Enter(): Thực thi một lần duy nhất khi thực thể bắt đầu bước vào trạng thái (dùng để khởi tạo dữ liệu, đổi hoạt ảnh).
 - + Update(): Liên tục thực thi ở mỗi khung hình để cập nhật logic của trạng thái đó (ví dụ: trạng thái Patrol sẽ liên tục tính toán tọa độ di chuyển tuần tra).
 - + Exit(): Thực thi một lần duy nhất trước khi rời bỏ trạng thái để dọn dẹp dữ liệu cũ.
2. Cơ chế liên kết điều kiện: Hệ thống liên tục kiểm tra các điều kiện môi trường trong hàm Update(). Ví dụ: Nếu khoảng cách giữa Quái vật và Người chơi nhỏ hơn tầm nhìn ($d < Range$), FSM sẽ kích hoạt lệnh chuyển dịch từ Patrol (Tuần tra) sang Chase (Đuổi theo).

2.5.3 Hướng mở rộng : HFSM, Behavior Tree

Mặc dù FSM rất trực quan và dễ cài đặt, cấu trúc này bộc lộ nhược điểm lớn khi quy mô trò chơi mở rộng, tạo ra hiện tượng "Bùng nổ trạng thái" – khi số lượng đường nối chuyển dịch giữa các trạng thái tăng lên theo hàm mũ, khiến mã nguồn cực kỳ rối rắm và khó gỡ lỗi. Để khắc phục, hai hướng tiếp cận nâng cao thường được áp dụng:

1. Hierarchical Finite State Machine: HFSM cho phép một trạng thái lớn chứa các trạng thái con bên trong nó. Ví dụ, trạng thái cha Combat (Chiến đấu) có thể chứa các trạng thái con là MeleeAttack (Đánh cận chiến) và RangedAttack (Tấn công xa). Nếu thực thể đang ở bất kỳ trạng thái con nào mà gặp điều kiện rút lui, hệ thống chỉ cần xử lý một đường chuyển dịch duy nhất từ trạng thái cha Combat về Evade (Né tránh), giảm thiểu tối đa các đường nối trùng lặp.

2. Behavior Tree: Là cấu trúc dịch chuyển hoàn toàn khỏi tư duy của FSM, tổ chức logic AI theo dạng cây phân cấp gồm các nút điều khiển (Selector, Sequence) và các nút lá thực thi hành động (Action Node). Behavior Tree tách biệt hoàn toàn điều kiện kiểm tra khỏi hành động, giúp lập trình viên dễ dàng tái sử dụng các module hành vi và mở rộng các hệ thống AI cực kỳ phức tạp mà không sợ bị giới hạn bởi các mối quan hệ chuyển dịch cứng nhắc của FSM.

2.6 Kỹ thuật mạng trong game real-time

Trong các trò chơi trực tuyến thời gian thực (như bắn súng, hành động, hoặc các mô phỏng nhiều người chơi), thách thức lớn nhất của tầng mạng là giải quyết độ trễ đường truyền (Network Latency / Ping). Nếu Client phải gửi lệnh điều khiển lên Server, chờ Server tính toán rồi gửi kết quả trả về thì người chơi sẽ cảm thấy nhân vật bị khựng lại (mất từ 50ms đến 200ms). Để mang lại trải nghiệm mượt mà, hệ thống bắt buộc phải áp dụng các kỹ thuật đồng bộ và giảm thiểu độ trễ nâng cao.

2.6.1 Mô hình Client-Server

Đề tài áp dụng mô hình Authoritative Server (Server làm chủ). Trong kiến trúc này:

1. Client chỉ đóng vai trò gửi các thao tác thô của người chơi (nhập từ bàn phím, chuột) lên Server, đồng thời nhận dữ liệu trạng thái thế giới từ Server về để hiển thị lên màn hình. Client không có quyền tự quyết định logic game (ví dụ: không được tự ý cộng tọa độ hay trừ máu quái vật).
2. Server là nơi nắm giữ "sự thật duy nhất" (Authority). Server nhận dữ liệu đầu vào của tất cả các Client, chạy logic mô phỏng trò chơi tại một tốc độ cố định (gọi là Tick rate, ví dụ 60 Ticks/giây), sau đó đóng gói trạng thái của toàn bộ thế giới game (Snapshot) và gửi ngược lại cho các Client. Mô hình này giúp ngăn chặn tuyệt đối các hành vi gian lận (Hack/Cheat) từ phía người chơi.

2.6.2 Client-side Prediction

Để xóa bỏ cảm giác giật lag khi chờ phản hồi từ Server, kỹ thuật Dự đoán phía Client được áp dụng.

1. Khi người chơi nhấn phím di chuyển sang phải, thay vì đứng yên chờ Server cho phép, Client sẽ ngay lập tức áp dụng logic di chuyển vào thực thể của mình để cập nhật vị trí mới trên màn hình ngay lập tức. Người chơi sẽ cảm nhận game phản hồi tức thì với độ trễ bằng 0.
2. Tuy nhiên, vị trí này tạm thời chỉ là một sự "dự đoán" từ phía Client và có thể bị thay đổi nếu Server từ chối ở các bước sau.

2.6.3 Input Buffering

Tốc độ truyền gói tin qua Internet thường không ổn định, dẫn đến hiện tượng trôi sụt độ trễ. Nếu Server xử lý các gói tin đầu vào ngay khi vừa nhận được, chuyển động của nhân vật sẽ bị giật cục do các gói tin đến không đều.

Kỹ thuật Input Buffering giải quyết vấn đề này bằng cách thiết lập một vùng đệm (Buffer) tại Server để lưu trữ các gói lệnh đầu vào của Client. Các gói lệnh này luôn đi kèm với một chỉ số khung thời gian (TickID). Server sẽ không xử lý ngay mà giữ lại một lượng nhỏ gói tin (ví dụ: đệm trước 2-3 Ticks), sau đó trích xuất dữ liệu đều đặn ở mỗi chu kỳ Tick của Server. Kỹ thuật này đánh đổi một khoảng độ trễ cực nhỏ để lấy sự mượt mà và tính tuần tự cho các chuyển động của thực thể.

2.6.4 Reconciliation và Rollback

Vì Client liên tục tự ý chạy trước (Prediction), sẽ có lúc kết quả dự đoán của Client bị lệch so với kết quả tính toán thực tế của Server (ví dụ: do Client bị quái vật cản đường mà Client chưa biết, hoặc do Server tính toán va chạm chính xác hơn). Khi Server gửi Snapshot chính thức về, Client phát hiện ra sự sai lệch này. Lúc này, cơ chế Reconciliation (Hòa giải) sẽ được kích hoạt:

1. Ghi nhận sai lệch: Client so sánh vị trí của thực thể tại một TickID trong quá khứ với vị trí chính thức mà Server vừa gửi về cho chính TickID đó.
2. Đảo ngược (Rollback): Nếu có sự sai lệch vượt ngưỡng cho phép, Client lập tức ghi đè (Force Reset) vị trí của thực thể về đúng vị trí mà Server chỉ định tại thời điểm quá khứ đó.
3. Mô phỏng lại (Replay): Client lấy toàn bộ các gói tin điều khiển (Input) mà người chơi đã bấm từ thời điểm quá khứ đó cho đến khung hình hiện tại (được lưu trong một mảng Local), lập tức tính toán chạy lại (Fast-forward) các System di chuyển một cách nhanh chóng để đưa nhân vật về đúng vị trí hiện tại hợp lệ.

Quá trình Rollback và Replay diễn ra ngay trong một khung hình duy nhất, giúp sửa lỗi vị trí một cách chính xác mà người chơi rất khó nhận ra bằng mắt thường.

CHƯƠNG 3 : THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng thể

Để hiện thực hóa một hệ thống xử lý thời gian thực hiệu năng cao, kiến trúc tổng thể của hệ thống được thiết kế dựa trên sự tách biệt hoàn toàn giữa các module chức năng. Trục xương sống của toàn bộ hệ thống là lõi ECS Core, nơi quản lý vòng đời của các thực thể và phân phối bộ nhớ phẳng cho các thành phần dữ liệu.

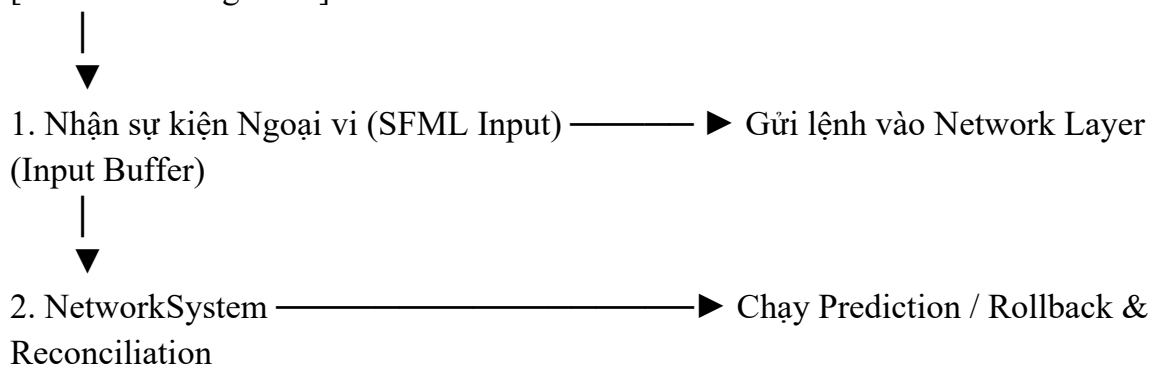
Vòng lặp trò chơi (Game Loop) đóng vai trò điều khiển trung tâm, kích hoạt các hệ thống logic (Systems) thực thi theo một thứ tự tuyến tính nghiêm ngặt trong mỗi

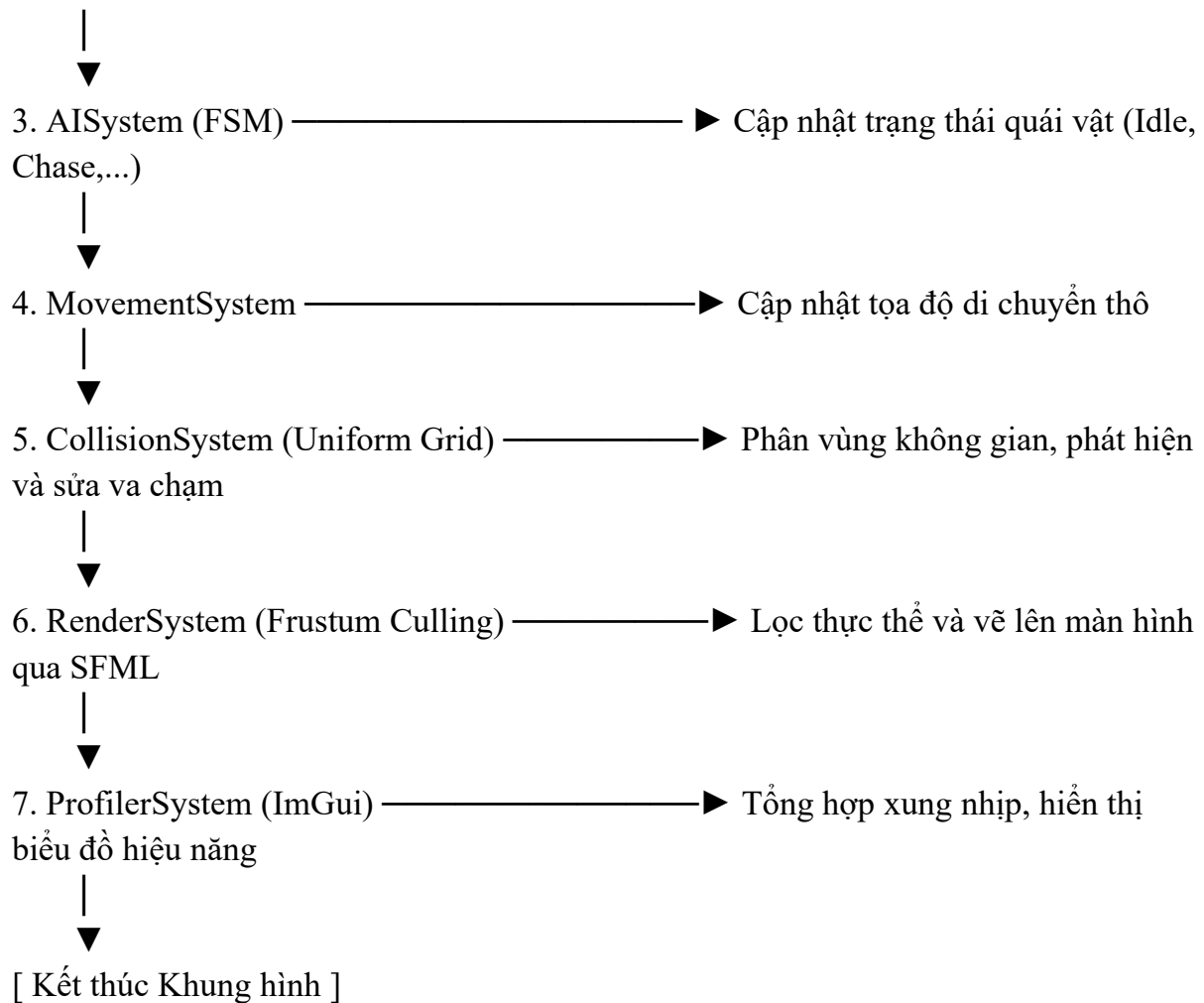
khung hình (Frame). Sơ đồ kiến trúc tổng thể của hệ thống được tổ chức theo các tầng xử lý như sau:

1. Tầng Phần cứng và Thư viện Cơ sở (Hardware & Foundation Layer): Nằm ở đáy hệ thống, bao gồm hệ điều hành, bộ nhớ RAM, và các thư viện liên kết ngoài như SFML (quản lý cửa sổ, vẽ đồ họa, bắt sự kiện thiết bị ngoại vi) và ImGui (hỗ trợ giao diện công cụ Profiler). Tầng này chịu trách nhiệm cung cấp tài nguyên thô cho Engine.
2. Tầng Lõi Hệ thống (ECS Core Layer): Gồm bộ ba quản lý cốt lõi: EntityManager (cấp phát thực thể), ComponentManager (quản lý bộ nhớ Sparse Set cho từng loại dữ liệu), và SystemManager (quản lý thứ tự chạy của logic). Tầng này đóng vai trò như một bộ điều phối trung tâm, đảm bảo dữ liệu luôn được sắp xếp liên tục trên bộ nhớ phẳng nhằm tối ưu hóa CPU Cache.
3. Tầng Hệ thống Logic và Thuật toán (Systems & Algorithms Layer): Nơi chứa các System nghiệp vụ cụ thể. Các System này lấy dữ liệu component từ ECS Core, kết hợp với các cấu trúc dữ liệu và thuật toán hỗ trợ để tính toán:
 - + NetworkSystem: Tương tác với tầng mạng giả lập, thực hiện dự đoán (Prediction) và hòa giải (Reconciliation).
 - + AISystem: Sử dụng mô hình FSM để cập nhật trạng thái hành vi cho các quái vật.
 - + MovementSystem: Xử lý cộng vận tốc vào vị trí một cách tuyến tính.
 - + CollisionSystem: Sử dụng cấu trúc Uniform Grid để truy vấn nhanh các thực thể lân cận và xử lý va chạm.
 - + RenderSystem: Áp dụng kỹ thuật Frustum Culling để lọc bỏ các thực thể ngoài màn hình trước khi gửi lệnh vẽ sang thư viện SFML.
 - + ProfilerSystem: Sử dụng các bộ đếm thời gian dạng Scope để đo đặc xung nhịp của toàn bộ các System khác và xuất dữ liệu ra bảng biểu ImGui.

Luồng vận hành tuần tự trong một chu kỳ (Tick/Frame) của kiến trúc tổng thể được thiết kế như sau:

[Bắt đầu Khung hình]





Nhờ cách thiết kế tách biệt và tuần tự này, hệ thống loại bỏ được hoàn toàn sự phụ thuộc lẫn nhau (Coupling) giữa các module. Dữ liệu đầu ra của System này sẽ trôi xuôi theo dòng chảy của vòng lặp để làm đầu vào cho System kế tiếp, giúp tối ưu hóa tối đa hiệu năng xử lý của CPU.

3.2 ECS Core

Bộ lõi ECS Core đóng vai trò là kiến trúc hạ tầng cho toàn bộ Game Engine, chịu trách nhiệm quản lý vòng đời của các thực thể, tổ chức lưu trữ dữ liệu tối ưu trên RAM và điều phối thứ tự thực thi của các mạch logic.

3.2.1 EntityManager - cấp phát và tái sử dụng EntityID

Trong hệ thống, Entity không phải là một đối tượng chứa dữ liệu mà chỉ là một số nguyên không dấu (uint32_t). Lớp EntityManager được thiết kế để quản lý tập hợp các định danh này với hai nhiệm vụ cốt lõi: cấp phát ID mới cho các thực thể vừa sinh ra và thu hồi, tái sử dụng các ID của thực thể đã bị hủy bỏ nhằm tránh hiện tượng tràn số nguyên (Integer Overflow).

- Cơ chế lưu trữ và cấp phát: EntityManager quản lý một biến đếm tuyến tính (`m_LivingEntityCount`) để đánh dấu ID lớn nhất từng được cấp phát và một hàng đợi (`std::queue<Entity> m_DestroyedEntities`) chứa các ID đã bị hủy.
- Giải thuật tạo Entity mới: Khi có yêu cầu tạo thực thể, hệ thống sẽ ưu tiên kiểm tra hàng đợi `m_DestroyedEntities`. Nếu hàng đợi không trống, một ID cũ sẽ được lấy ra (Pop) và tái sử dụng. Nếu hàng đợi trống, hệ thống sẽ lấy giá trị biến đếm hiện tại làm ID mới và tăng biến đếm lên một đơn vị. Quy trình này đạt độ phức tạp thời gian lý tưởng là $O(1)$.
- Quản lý Signature (Mặt nạ bit): Để xác định một thực thể đang sở hữu những Component nào, EntityManager quản lý một mảng phẳng các chuỗi bit (`std::vector<Bitset> m_Signatures`), trong đó chỉ mục của mảng chính là EntityID. Mỗi loại Component trong game sẽ ứng với một vị trí bit cố định. Khi thực thể được thêm hoặc xóa một Component, bit tương ứng sẽ được bật (1) hoặc tắt (0).

3.2.2 Memory Pool với SparseSet

ComponentManager chịu trách nhiệm quản lý bộ nhớ cho toàn bộ dữ liệu thô của trò chơi. Để bề bầy hoàn toàn sự phân mảnh bộ nhớ của OOP, hệ thống áp dụng giải pháp thiết kế một vùng lưu trữ tập trung (Memory Pool) độc lập cho từng loại Component riêng biệt dựa trên cấu trúc dữ liệu Sparse Set.

- Cấu trúc phân tách bộ nhớ: Thay vì tạo một mảng chứa các đối tượng hỗn hợp, hệ thống tổ chức bộ nhớ theo dạng `ComponentStorage<T>`, trong đó T là kiểu dữ liệu của Component (ví dụ: `TransformComponent`, `ColliderComponent`). Mỗi `ComponentStorage<T>` sở hữu một bộ Sparse Set gồm:
 - `m_SparseArray`: Mảng thưa dùng EntityID làm chỉ mục để tra cứu vị trí dữ liệu.
 - `m_DenseArray`: Mảng dày đặc lưu danh sách các EntityID xếp liền kề nhau.
 - `m_ComponentArray`: Mảng phẳng lưu trữ trực tiếp các struct dữ liệu thuần kiểu T.
- Đồng bộ chỉ mục liên tục: Mảng dày và mảng thành phần được thiết kế để quản lý chỉ mục song song 1:1. Khi một System duyệt qua mảng dữ liệu để tính toán, nó chỉ cần lướt tuyến tính trên `m_ComponentArray`. Do dữ liệu kiểu T được xếp sát nhau trên các ô nhớ liên tiếp, CPU có thể kích hoạt cơ chế nạp trước dữ liệu vào các tầng Cache (Cache Prefetching), giúp loại bỏ hoàn toàn các chu kỳ chết do trượt bộ nhớ đệm.

3.2.3 SystemManager

Các System chứa mã nguồn xử lý logic nhưng hoàn toàn không lưu trữ trạng thái dữ liệu của thực thể. Lớp SystemManager được thiết kế theo tư duy lập trình meta-

programming (Meta-programming) nhằm quản lý vòng đời của các hệ thống này, đảm bảo tính tuần tự và tối ưu hóa hiệu năng tuyệt đối tại thời điểm biên dịch (Compile-time).

Quản lý hệ thống bằng cấu trúc bộ ba tĩnh (Static Tuple Processing): Thay vì sử dụng các danh sách liên kết động (như `std::vector` chứa con trỏ) gây phân mảnh bộ nhớ và tốn chi phí duyệt tìm lúc runtime, `SystemManager` đóng gói toàn bộ các lớp `System` vào một cấu trúc `std::tuple<Systems...>`. Giải pháp này cho phép hệ thống sử dụng kỹ thuật giải nén biểu thức (Fold Expression) kết hợp với `std::apply`. Cú pháp triển khai giúp trình biên dịch tự động mở phẳng vòng lặp (Loop unrolling), chuyển các thao tác gọi hàm vòng đời thành các lời gọi trực tiếp tuần tự trong mã máy với chi phí bằng không (Zero-overhead).

Quản lý thứ tự thực thi tuần tự cố định (Compile-time Dependency): Do hệ thống vận hành trên kiến trúc đơn luồng (Single-threading) để đảm bảo tính đồng bộ thời gian thực, thứ tự kích hoạt hàm `update()` của các `System` quyết định tính đúng đắn của toàn bộ logic trò chơi. `SystemManager` thiết lập chuỗi xử lý tuần tự nghiêm ngặt dựa trên chính thứ tự khai báo kiểu dữ liệu trong tham số `Template`. Chuỗi vòng lặp hệ thống (Game Loop Flow) được quy định chặt chẽ như sau:

1. Các hệ thống tiếp nhận và xử lý dữ liệu đầu vào hoặc trạng thái nền tảng (`NetworkSystem`, `AISystem`).
2. Các hệ thống thực hiện biến đổi vật lý, động học và giải quyết va chạm không gian (`MovementSystem`, `CollisionSystem`).
3. Hệ thống cuối cùng là kết xuất đồ họa (`RenderSystem`) để phản hồi trực quan toàn bộ trạng thái dữ liệu đã qua xử lý lên màn hình thông qua thư viện `SFML`.

3.3 Thiết kế Sparse Set

Để hiện thực hóa Memory Pool cho `ComponentManager` trong lõi ECS Core, cấu trúc dữ liệu Sparse Set được thiết kế dưới dạng lớp khuôn mẫu `SparseSet<T>`, với `T` là kiểu dữ liệu thô của `Component`. Cấu trúc này tối ưu hóa bộ nhớ nhằm đạt tốc độ xử lý $O(1)$ cho mọi thao tác cơ bản mà không gánh chi phí quản lý dư thừa.

3.3.1 Cấu trúc mảng thành phần

`SparseSet<T>` quản lý nội bộ ba mảng dữ liệu phẳng động (sử dụng `std::vector` nội bộ và kiểm soát chặt chẽ cơ chế cấp phát):

1. `m_Sparse` (Mảng thưa): Sử dụng trực tiếp Entity ID làm chỉ mục (Index). Giá trị lưu tại `m_Sparse[entity]` trỏ đến chỉ mục tương ứng của thực thể đó trong Mảng Dày Đặc. Ô trống được lấp đầy bằng giá trị mặc định `INVALID_INDEX`.

2. `m_Dense` (Mảng dày đặc): Lưu trữ liên tiếp danh sách các Entity ID thực tế đang sở hữu Component, đảm bảo bộ nhớ không có khoảng trống từ chỉ mục 0 đến `mSize - 1`.
3. `m_Instances` (Mảng thực thể Component): Lưu trữ các struct dữ liệu thô (SoA - Structure of Arrays). Chỉ mục của mảng này đồng bộ song song tỉ lệ 1:1 với `m_Dense`.

3.3.2 Thiết kế các thuật toán cốt lõi

1. Hàm Kiểm tra Sở hữu (`Has(Entity entity)`): `mSparse[entity] != INVALID_INDEX`
2. Hàm Chèn mới (`Insert(Entity entity, T component)`):
 Nếu thực thể đã sở hữu Component, hệ thống chỉ ghi đè dữ liệu tại `m_Instances`. Nếu là thực thể mới, thuật toán thực hiện với độ phức tạp $O(1)$:
 - + Đẩy `entity` vào cuối mảng `m_Dense`.
 - + Đẩy dữ liệu `component` vào cuối mảng `m_Instances`.
 - + Cập nhật chỉ mục định vị: `m_Sparse[entity] = m_Size`.
 - + Tăng kích thước `m_Size` thêm 1.
3. Hàm Xóa bỏ (`Remove(Entity entity)`):
 Áp dụng kỹ thuật Swap and Pop (Hoán đổi và Thu hẹp) để xóa phần tử ở giữa mảng dữ liệu phẳng trong $O(1)$ mà không làm mất tính liên tục của bộ nhớ:
 1. Lấy chỉ mục phần tử cần xóa: `idx = m_Sparse[entity]`.
 2. Lấy ID của thực thể cuối cùng trong mảng dày: `last_entity = m_Dense[m_Size - 1]`.
 3. Hoán đổi phần tử cần xóa với phần tử cuối cùng trên cả `m_Dense` và `m_Instances`.
 4. Cập nhật lại mảng thừa cho thực thể cuối về vị trí mới:
`m_Sparse[last_entity] = idx`.
 5. Đặt lại giá trị trống cho thực thể vừa xóa: `m_Sparse[entity] = INVALID_INDEX`.
 6. Thu hẹp kích thước mảng (`pop_back`) và giảm `m_Size` đi 1.

3.4 Spatial Partitioning System

Hệ thống phân vùng không gian đóng vai trò là bộ lọc vật lý cho Game Engine. Thay vì kiểm tra va chạm mù quáng, hệ thống này tổ chức lại cấu trúc không gian của màn chơi, giúp các System giới hạn phạm vi tính toán trong một vùng cục bộ nhỏ nhất có thể.

3.4.1 Lựa chọn cấu trúc (Uniform Grid)

Sau khi phân tích các đặc tính vận hành của game 2D hiệu năng cao, Uniform Grid (Lưới đều) được lựa chọn làm cấu trúc phân vùng chính cho hệ thống thay vì Quadtree. Quyết định thiết kế này dựa trên các luận điểm kỹ thuật sau:

1. Tối ưu hóa tần suất cập nhật $O(1)$: Trong kịch bản game giả lập bảo đạn hoặc quái vật di chuyển liên tục, vị trí của các thực thể thay đổi ở mỗi khung hình. Với Quadtree, việc thực thể di chuyển xuyên vùng bắt buộc hệ thống phải thực hiện giải thuật đệ quy để hủy, gộp hoặc phân tách lại các nút cây, gây áp lực rất lớn cho CPU. Đối với Uniform Grid, thao tác cập nhật khi thực thể chuyển ô lưới chỉ đơn thuần là tính lại chỉ mục qua phép toán chia số nguyên và dịch chuyển ID trong mảng phẳng với độ phức tạp cố định $O(1)$.
2. Thiết kế cấu trúc lưu trữ phẳng: Hệ thống quản lý lưới thông qua một mảng một chiều phẳng duy nhất đại diện cho không gian hai chiều, được cấu trúc dưới dạng:

3.4.2 Tích hợp vào Collision Detection System trong ECS

Quy trình tích hợp Uniform Grid vào hệ thống phát hiện va chạm (CollisionSystem) của ECS được thiết kế tách biệt thành hai giai đoạn rõ rệt trong mỗi chu kỳ cập nhật logic:

1. Giai đoạn 1: Làm mới và Đăng ký Lưới (Grid Rebuild):
Vào đầu mỗi khung hình, CollisionSystem sẽ xóa sạch dữ liệu ID cũ trong mảng `m_GridCells`. Sau đó, System truy vấn toàn bộ các Entity sở hữu cập thành phần TransformComponent (chứa vị trí) và ColliderComponent (chứa hộp móng va chạm AABB). Hệ thống duyệt tuyến tính qua mảng phẳng này, tính toán chỉ mục CellIndex dựa trên tọa độ hiện tại của thực thể và đẩy (push_back) Entity ID vào ô lưới tương ứng.
2. Giai đoạn 2: Truy vấn và Kiểm tra va chạm cục bộ:
Khi tiến hành tính toán va chạm cho một Entity A, CollisionSystem không duyệt qua toàn bộ thế giới game. Nó xác định ô lưới hiện tại của A và chỉ truy cập vào ô đó cùng với 8 ô lân cận xung quanh (tổng cộng 9 ô lưới).

Hệ thống tiến hành lặp kép để kiểm tra va chạm hình học (AABB vs AABB) giữa A và các Entity nằm trong tập hợp 9 ô này. Thiết kế này giúp giảm độ phức tạp từ $O(N^2)$ xuống mức tiệm cận $O(N)$ trong điều kiện thực thể phân bố tương đối đều, loại bỏ hoàn toàn các chu kỳ tính toán lãng phí cho các thực thể nằm cách xa nhau.

3.5 AI System - Finite State Machine

Hệ thống AI xử lý hành vi của thực thể trong Engine được thiết kế dựa trên sự kết hợp giữa tính linh hoạt của mô hình máy trạng thái hữu hạn (FSM) và cấu trúc phẳng tối ưu cache của kiến trúc ECS.

3.5.1 Thiết kế State Machine hướng sự kiện (Event-Driven FSM)

Để đảm bảo tính tương thích tuyệt đối với kiến trúc ECS và tách biệt hoàn toàn logic xử lý khỏi trạng thái lưu trữ, hệ thống thiết kế cơ chế máy trạng thái dựa trên mô hình

Phản ứng sự kiện (Event-driven) kết hợp đóng gói thành phần dữ liệu thông qua giao diện trừu tượng IState.

- Giao diện mẫu IState: Không sử dụng các hàm vòng đời truyền thống (Enter/Update/Exit) vốn gây phân mảnh logic, IState định nghĩa một cơ chế tiếp nhận và xử lý biến cố duy nhất thông qua hàm thành viên thuần ảo:
- C++ : `virtual IComponentBundle* onEvent(SceneCombat* scene, int entityId, UnitEventKind event, int targetId) const = 0;`
 - Hàm này nhận đầu vào là ngữ cảnh trận đấu (SceneCombat), mã thực thể (entityId), loại biến cố mạng/môi trường (UnitEventKind) và thực thể mục tiêu liên quan (targetId). Khi có sự kiện kích hoạt, trạng thái hiện tại sẽ tính toán và trả về một gói chứa các Component mới (IComponentBundle) nhằm thay đổi hành vi của thực thể trực tiếp trong bộ nhớ ECS Core.
 - Phân định tập hợp trạng thái (UnitStateKind): Hệ thống chuẩn hóa hành vi của các đơn vị AI trong trò chơi thành ba trạng thái cốt lõi thông qua lớp kiểu dữ liệu cấu trúc UnitStateKind:
 - Search (Tìm kiếm/Tuần tra): Trạng thái chủ động quét mục tiêu trên bản đồ.
 - Combat (Giao tranh): Trạng thái tập trung tấn công và bám đuổi mục tiêu.
 - Retreat (Rút lui): Trạng thái tháo chạy khi gặp nguy hiểm để bảo toàn đơn vị.

3.5.2 Cơ chế chuyển dịch trạng thái qua Component Bundle

Quy trình dịch chuyển trạng thái (State Transition) không thực hiện bằng cách thay đổi con trỏ trạng thái cục bộ của Object mà được vận hành thông qua cơ chế Thay đổi cấu trúc thực thể (Entity Mutation) trong ECS.

- Tiếp nhận Biến cố (UnitEventKind): Hệ thống AI liên tục lắng nghe các tín hiệu thay đổi từ môi trường game được phân loại rõ ràng qua UnitEventKind bao gồm: EnemySpotted (Phát hiện địch), TargetLost (Mất dấu mục tiêu), HealthLow (Thấp máu), và Safe (Đã an toàn).
- Quy trình chuyển dịch trạng thái tuần tự: Khi một sự kiện được gửi đến thực thể, trạng thái hiện tại (ví dụ: SearchState) sẽ kiểm tra điều kiện thông qua cấu trúc rẽ nhánh switch-case. Nếu điều kiện chuyển dịch thỏa mãn, quy trình biến đổi cấu trúc diễn ra như sau:
 1. Trạng thái Search sang Combat: Khi nhận biến cố EnemySpotted với một targetId hợp lệ, SearchState lập tức kích hoạt hàm tạo gói dữ liệu

giao tranh `createCombatBundle()`, nạp các thành phần điều khiển tấn công vào thực thể.

2. Trạng thái Combat sang Search hoặc Retreat: Khi đang giao tranh, nếu nhận biến cố `TargetLost`, hệ thống sinh ra `createSearchBundle()` để AI quay lại trạng thái tìm kiếm. Nếu nhận biến cố `HealthLow`, hệ thống lập tức sinh ra `createRetreatBundle()` nhằm ép thực thể kích hoạt logic rút lui.
3. Trạng thái Retreat sang Search: Khi AI rút lui an toàn và nhận biến cố `Safe`, `RetreatState` giải phóng trạng thái cũ, sinh ra `createSearchBundle()` đưa thực thể về vòng lặp tuần tra ban đầu.

Toàn bộ quá trình chuyển dịch này trả về một con trỏ dữ liệu thành phần. Nếu điều kiện không thỏa mãn hoặc trạng thái đang trong hàng đợi xử lý dữ liệu (`pending = true`), hàm trả về `nullptr`, giữ nguyên trạng thái hiện tại của thực thể một cách an toàn và nhất quán.

3.5.3 Tích hợp FSM vào ECS

Thách thức lớn nhất khi đưa FSM vào kiến trúc ECS là giải quyết xung đột triết lý: FSM truyền thống ràng buộc chặt chẽ giữa trạng thái và hành vi (OOP), trong khi ECS yêu cầu tách rời hoàn toàn dữ liệu và logic. Hệ thống giải quyết triệt để vấn đề này bằng cách phân tách FSM thành hai tầng dữ liệu và logic riêng biệt:

- Thiết kế thành phần dữ liệu trạng thái (`CAIState`):
Trạng thái của bộ não AI được lưu trữ dưới dạng một Component thuần túy, đóng vai trò là dữ liệu thô nằm trong mảng phẳng liên kề của `ComponentStorage`:

```
struct CAIState {  
    UnitStateKind currentKind;    // Định danh trạng thái hiện tại  
    (Search, Combat, Retreat)  
    int targetEntityId;           // ID của thực thể mục tiêu (-1 nếu  
    không có)  
    bool isPending;              // Đánh dấu trạng thái đang chờ xử lý  
    đồng bộ  
};
```

- Thành phần này tuyệt đối không chứa mã nguồn xử lý mà chỉ phục vụ mục đích lưu trữ dữ liệu để các System truy vấn.
- Thiết kế hệ thống điều phối (`AISystem`):
`AISystem` đóng vai trò là tầng logic, đăng ký một mặt nạ bit (Signature) yêu cầu các thực thể phải sở hữu cặp thành phần `CAIState` và `CTransform`. Thay vì gọi hàm cập nhật logic AI cho từng thực thể ở mỗi khung hình một cách tuần tự

gây lãng phí CPU, AISystem vận hành theo cơ chế Kích hoạt dựa trên biến cố (Event-Triggered)

1. Tiếp nhận biến cố: Khi thế giới game phát sinh các sự kiện môi trường hoặc mạng (ví dụ: CollisionSystem phát hiện mục tiêu lọt vào tầm quét sau đó sinh sự kiện EnemySpotted), các sự kiện này được đẩy tập trung về cho AISystem.
2. Tra cứu trạng thái tĩnh: AISystem duyệt qua các thực thể thỏa mãn, lấy ra dữ liệu currentKind từ CAIState. Từ định danh này, hệ thống ánh xạ sang đối tượng trạng thái tĩnh tương ứng (SearchState, CombatState, hoặc RetreatState) trong bộ nhớ dùng chung.
3. Thực thi Mutation và áp dụng Bundle: Hệ thống gọi hàm onEvent() của trạng thái đó. Nếu điều kiện chuyển dịch thỏa mãn, một con trỏ IComponentBundle chứa tập hợp các Component hành vi mới sẽ được trả về. AISystem sẽ ra lệnh cho SceneCombat giải phóng các Component cũ và nạp gói Component mới vào thực thể.

Giải pháp tích hợp này giúp giữ cho cấu trúc bộ nhớ của ECS luôn ở dạng phẳng, tận dụng tối đa tốc độ duyệt tuyến tính của Sparse Set, đồng thời biến các hành vi AI phức tạp thành các thao tác thêm/xóa dữ liệu (Mutation) có chi phí tính toán cực thấp.

3.6 Network Layer

Tầng mạng trong hệ thống không sử dụng các socket phân cứng truyền thống mà được thiết kế dưới dạng một Hệ thống giả lập mạng trong bộ nhớ (In-memory Network Link Simulator) thông qua lớp SceneNet. Mục tiêu của thiết kế này là mô phỏng, trực quan hóa và đánh giá chính xác các thuật toán đồng bộ, dự đoán trạng thái và bù trễ thời gian thực.

3.6.1 Kiến trúc Server - Client giả lập

Hệ thống tích hợp cả hai thành phần Server và Client vào cùng một không gian bộ nhớ nhưng hoạt động trên hai luồng logic độc lập logic (SNet::ClientGame và SNet::ServerGame):

- Cấu trúc dữ liệu Gói tin (SNet::Package): Là đơn vị truyền dẫn cốt lõi, đóng gói toàn bộ thông tin bao gồm: số hiệu khung hình (frame), tọa độ thực thể (pos), cấu hình nút bấm đầu vào (input), cấu trúc trạng thái (state) và bộ đếm thời gian trễ tích lũy (time).
- Mô phỏng độ trễ vật lý (Lag Simulation): Hệ thống thiết lập hai hằng số thời gian lag_client (độ trễ gửi từ Client lên Server) và lag_server (độ trễ gửi từ Server về Client). Các gói tin khi phát đi sẽ không đến đích ngay lập tức mà được giữ lại trong danh sách liên kết sending. Hệ thống liên tục tích lũy biến thời gian delta m_time.lag. Chỉ khi thời gian tích lũy vượt ngưỡng lag_client

hoặc `lag_server`, gói tin mới được giải phóng sang danh sách `received` của đối phương.

3.6.2 Input Buffering và Cơ chế điều tốc mạng (Client Adaptive Speed)

Để đối phó với hiện tượng mất đồng bộ và trôi sụt gói tin (Jitter), hệ thống triển khai bộ đệm đầu vào tại Server (`m_server.received`) kết hợp với giải thuật tự động điều tốc tại Client:

- Cơ chế lưu đệm tại Server: Gói lệnh điều khiển của Client được đẩy vào hàng đợi của Server. Server chỉ xử lý khi khung hình của gói tin hợp lệ với mốc thời gian hiện tại của Server.
- Thuật toán tự động điều tốc (Client Adaptive Speed): Hệ thống thiết kế một cơ chế điều khiển vận tốc thời gian của Client (`m_client.speed`) dựa trên số lượng gói tin còn tồn đọng trong bộ đệm của Server (`buffer = m_server.received.size()`):
 - Nếu `buffer == 0` (Server đang thiếu dữ liệu do mạng chậm): Client tự động tăng tốc lên $1.2f$ để gửi bù input.
 - Nếu `buffer >= 3` (Ứng đọng dữ liệu): Client giảm tốc xuống $0.4f$ để chờ Server tiêu thụ hết bộ đệm.
 - Ngược lại, Client duy trì tốc độ tiêu chuẩn $1.0f$. Cơ chế này giúp ổn định dòng chảy dữ liệu đầu vào mà không cần can thiệp sâu vào luồng xử lý phần cứng.

3.6.3 Client-side Prediction và Danh sách kiểm thử (`state_pendings`)

Để triệt tiêu độ trễ cảm nhận từ phía người chơi, Client không đợi phản hồi từ Server mà chủ động thực hiện dự đoán trước:

- Chạy trước Frame: Client liên tục gia tăng chỉ số `m_client.frame` và tính toán vị trí dự đoán mới dựa trên vận tốc tuyến tính và khoảng cách cập nhật cố định (`gap_update`).
- Lưu trữ hàng đợi kiểm thử (`state_pendings`): Khi tạo ra một khung hình mới, Client đóng gói một cấu trúc trạng thái giả định `SNet::State` và đẩy vào danh sách `m_client.state_pendings`. Biến value trong cấu trúc này đại diện cho trạng thái logic mà Client tin rằng nó sẽ đúng sau khi Server xử lý.

3.6.4 Reconciliation và Xử lý sai lệch trạng thái (State Corroboration)

Khi Server xử lý xong một Tick logic, nó đóng gói và gửi một gói tin chứa trạng thái chuẩn (`pkg.state`) về cho Client. Quy trình đối soát diễn ra liên tục tại hàm `update()` của Client:

- Lọc bỏ dữ liệu lỗi thời: Client liên tục loại bỏ các trạng thái cũ trong danh sách `state_pendings` có chỉ số frame nhỏ hơn chỉ số khung hình mà Server vừa gửi về nhằm giải phóng bộ nhớ.
- Kiểm tra và Hòa giải sai lệch (Reconciliation): Client lấy trạng thái dự đoán cũ ở đầu danh sách `state_pendings` đối sánh trực tiếp với trạng thái chính thức của Server (`pkg.state`):
 - Trường hợp trùng khớp (`predicted_state.value == pkg.state.value`): Biến `match` được gán bằng 1 (Hiển thị màu Magenta trên màn hình). Hệ thống xác định Client đã dự đoán chính xác.
 - Trường hợp sai lệch (`predicted_state.value != pkg.state.value`): Hệ thống phát hiện Client dự đoán sai (Ví dụ: Do tác động từ các yếu tố ngẫu nhiên mạng ngầm định). Client lập tức thực hiện quy trình hòa giải bằng cách: Ép lại giá trị hiện tại về giá trị chuẩn của Server (`predicted_state.value = pkg.state.value`), sau đó duyệt dọc qua toàn bộ các trạng thái còn lại trong `state_pendings` để tính toán lại toàn bộ chuỗi trạng thái (`next_it->value = it->value + 1`) và đánh dấu `match = 0` (Hiển thị màu đỏ) báo hiệu quá trình sửa sai đang diễn ra.

3.7 Profiler

Để đảm bảo Game Engine vận hành ổn định ở mức tốc độ khung hình cao và kịp thời phát hiện các điểm nghẽn kỹ thuật (bottlenecks), đề tài triển khai một hệ thống đo đặc hiệu năng (Profiler) gọn nhẹ dựa trên cơ chế bắt công cụ (Instrumentation). Hệ thống này ghi nhận chính xác thời gian thực thi của từng hàm, từng phạm vi mã nguồn (scope) theo thời gian thực và xuất dữ liệu ra định dạng JSON chuẩn. Dữ liệu này sau đó được trực quan hóa trực tiếp thông qua công cụ `chrome://tracing` hoặc `speedscope.app` trên trình duyệt Chrome.

CHƯƠNG 4 : TRIỂN KHAI VÀ CÀI ĐẶT

4.1 Môi trường phát triển

Để hiện thực hóa kiến trúc Game Engine hướng dữ liệu với hiệu năng tối ưu, việc cấu hình một môi trường phát triển (Development Environment) đồng bộ và gọn nhẹ là yếu tố tiên quyết. Hệ thống ưu tiên sử dụng các công cụ mã nguồn mở, hỗ trợ biên dịch chéo và quản lý mã nguồn chặt chẽ nhằm đảm bảo tính ổn định của mã nguồn C++ trên các nền tảng khác nhau.

4.2 Cài đặt ECS Core

4.2.1 EntityManager

Lớp `EntityManager` đóng vai trò là bộ quản lý vòng đời (Lifecycle Manager) của toàn bộ các thực thể (Entity) trong trò chơi. Để giải quyết triệt để vấn đề phân mảnh bộ nhớ và tránh hiện tượng xung đột dữ liệu khi các System thêm/xóa Entity trực tiếp trong

chu kỳ cập nhật, EntityManager được cài đặt dựa trên hai kỹ thuật cốt lõi: Quản lý ID bằng cơ chế Bitmask tích hợp Ngăn xếp (std::stack) và Trì hoãn thao tác qua Bộ đệm lệnh (Command Buffer)

a) Định nghĩa cấu trúc dữ liệu và Giao diện quản lý

```
#ifndef ENTITYMANAGER_HPP
#define ENTITYMANAGER_HPP

#include "ComponentBundle.hpp"
#include "MemoryPool.hpp"
#include <stack>
#include <vector>
#include <memory>

struct EntityManager
{
    MemoryPool                m_pool;           // Bộ nhớ phẳng lưu trữ
    Component
        std::vector<int>      command_buffer_remove; // Bộ đệm trì hoãn
        xóa Entity
        std::vector<std::shared_ptr<IComponentBundle>> command_buffer_add; // Bộ
        đệm trì hoãn thêm Entity
        std::vector<size_t>    marks;           // Bitmask đánh dấu thực
        thể đang hoạt động

    int      maxId = 0; // Chỉ số ID lớn nhất từng được cấp phát
    std::stack<int> m_freeId; // Ngăn xếp chứa các ID đã bị xóa để tái sử dụng

    int  getID();
    void add(std::shared_ptr<IComponentBundle> &components);
    void remove(int entityID);
    void pushID(int entityID);
    void update();

    MemoryPool& pool();
    const std::vector<int>& getEntities() const;
};

#endif
```


b) Cài đặt giải thuật cấp phát và Tái sử dụng Thực thể (ID Allocation & Recycle)
Hệ thống quản lý ID dưới dạng các số nguyên (int). Nhằm tiết kiệm tài nguyên, khi một thực thể bị hủy, ID của nó không bị xóa bỏ vĩnh viễn mà được đẩy vào ngăn xếp `m_freeId` để cấp phát lại cho các thực thể sinh ra sau. Đồng thời, một mảng `marks` (`std::vector<size_t>`) đóng vai trò làm Bitmask để kiểm tra trạng thái tồn tại của Entity với chi phí tiệm cận $O(1)$ thông qua các phép toán bitwise trên CPU:

```
int EntityManager::getID()
{
    // Nếu không còn ID trống trong ngăn xếp, tiến hành cấp phát ID mới tăng dần
    if (m_freeId.empty())
    {
        // Tính toán số lượng phân đoạn (chunk) cần thiết dựa trên kích thước bit của
        // hệ thống
        int maxChunk = maxId / (8 * sizeof(size_t));
        while ((int)marks.size() < maxChunk + 1)
            marks.push_back(0); // Mở rộng mảng bitmask dynamic

        return maxId++;
    }

    // Tái sử dụng ID từ các thực thể đã bị hủy bỏ trước đó
    int id = m_freeId.top();
    m_freeId.pop();
    return id;
}
```

c) Cài đặt cơ chế Trì hoãn thao tác (Deferred Commands)

Trong kiến trúc ECS, việc trực tiếp thêm hoặc xóa các thành phần của một Entity ngay khi System đang duyệt mảng phẳng sẽ làm thay đổi cấu trúc bộ nhớ vật lý, dẫn đến lỗi sập chương trình (Dangling Pointer / Iterator Invalidation). Do đó, hàm `add` và `remove` được triển khai theo cơ chế ghi nhận lệnh vào hàng đợi để xử lý đồng bộ sau:

```
void EntityManager::add(std::shared_ptr<IComponentBundle> &components)
{
    int entityID = getID(); // Lấy ID hợp lệ (mới hoặc tái sử dụng)
```

```

    int chunk = entityID / (8 * sizeof(size_t)); // Xác định vị trí phần tử trong vector
marks
    int offset = entityID % (8 * sizeof(size_t)); // Xác định vị trí bit cụ thể trong
chunk

    marks[chunk] |= size_t(1) << offset; // Đánh dấu bit = 1: Thực thể bắt đầu hoạt
động
    components->entityID = entityID;

    command_buffer_add.push_back(components); // Đẩy vào hàng đợi chờ xử lý tập
trung
}

void EntityManager::remove(int entityID)
{
    int chunk = entityID / (8 * sizeof(size_t));
    int offset = entityID % (8 * sizeof(size_t));

    // Kiểm tra xem thực thể có đang hoạt động hay không thông qua phép toán AND
bit
    if ((marks[chunk] >> offset) & size_t(1))
    {
        marks[chunk] &= ~(size_t(1) << offset); // Hạ bit về 0: Đánh dấu đã xóa khỏi
logic game
        command_buffer_remove.push_back(entityID); // Đẩy ID vào hàng đợi chờ
giải phóng bộ nhớ
    }
}

```

d) Cài đặt hàm cập nhật đồng bộ chu kỳ (update)

```

void EntityManager::update()
{
    // Bước 1: Xử lý xóa triệt để các thực thể đã được đánh dấu hủy
    for (int &entityID : command_buffer_remove)
    {
        m_pool.remove(entityID); // Giải phóng toàn bộ Component của entity khỏi
MemoryPool
    }
}

```

```

    pushID(entityID);    // Đưa ID này vào ngăn xếp m_freeId để tái sử dụng ở
    khung hình sau
}

// Bước 2: Xử lý khởi tạo chính thức các thực thể mới
for (auto &components : command_buffer_add)
{
    m_pool.add(components->entityID); // Đăng ký ID vào bộ lưu trữ phẳng
    components->switchState(m_pool); // Giải nén Bundle để nạp các Component
    tương ứng vào Pool
}

// Bước 3: Làm rỗng bộ đệm lệnh, chuẩn bị cho chu kỳ tính toán tiếp theo
command_buffer_add.clear();
command_buffer_remove.clear();
}

```

4.2.2 ComponentStorage

a) Kiến trúc phân trang Paged Sparse Set

Để tránh việc mảng thưa (sparse) bị phình to vô hạn khi cấu trúc ID của Entity tăng cao, hệ thống triển khai cơ chế Phân trang (Paging) với cấu hình hằng số `MAX_PAGE_SIZE = 2000`. Bộ lưu trữ được định nghĩa trong file tiêu đề như sau:

```

#pragma once
#include <vector>

#define MAX_PAGE_SIZE 2000

template <typename T>
class SparseSet
{
    std::vector<int> slots;           // Đếm số lượng slot trống trong mỗi trang để giải
    phóng bộ nhớ
    std::vector<std::vector<int>>> sparse; // Mảng thưa đa tầng (Phân trang mảng hai
    chiều)
    std::vector<T> dense;           // Mảng đặc lưu trữ Component liên tiếp trên bộ
    nhớ RAM
}

```

```

    std::vector<int> indexs;          // Ánh xạ ngược từ vị trí mảng dense về lại Entity
ID

public:
    using Type = T;
    bool has(int ID);
    void add(int ID, T &&component);
    T &get(int ID);
    void remove(int ID);
    void clear();

    std::vector<T> &view();
    const std::vector<int> &viewIndex() const;
};
#include "Sparsset.inl"

```

b) Thuật toán kiểm tra và Cấp phát động theo trang (Page Allocation)

Khi truy vấn một Entity ID, hệ thống thực hiện toán tử chia logic để tìm ra trang (page) và vị trí tương đối (offset). Nếu thực thể vượt cấu trúc bộ nhớ hiện tại, hệ thống tự động co giãn (push_back) và gán giá trị mặc định -1 (biểu thị chưa có Component nào được nạp):

```

template <typename T>
inline bool SparseSet<T>::has(int ID)
{
    int page = ID / MAX_PAGE_SIZE;
    int offset = ID % MAX_PAGE_SIZE;

    // Tự động mở rộng danh sách quản lý trang nếu ID vượt ngưỡng hiện tại
    while ((int)sparse.size() < page + 1)
        sparse.push_back(std::vector<int>());
    while ((int)slots.size() < page + 1)
        slots.push_back(MAX_PAGE_SIZE);

    // Khởi tạo trang mới nếu trang này đang trống
    if (sparse[page].empty())
        sparse[page].assign(MAX_PAGE_SIZE, -1);
}

```

```
return sparse[page][offset] != -1;
}
```

c) Cài đặt giải thuật Thêm và Truy xuất dữ liệu với chi phí $O(1)$

Thao tác Thêm (add): Thành phần Component mới luôn được đẩy vào cuối mảng dense liên mạch, sau đó cập nhật chỉ số mảng này vào `sparse[page][offset]`. Do đó bộ nhớ lưu trữ thực tế luôn được xếp khít nhau, tối ưu hóa tối đa hiện tượng Cache Locality khi các System duyệt mảng tuyến tính.

Thao tác Truy xuất (get): Cho phép truy cập trực tiếp vào vùng nhớ dữ liệu thông qua hai lần nhảy con trỏ index, hoàn toàn không sử dụng vòng lặp.

```
template <typename T>
inline void SparseSet<T>::add(int ID, T &&component)
{
    int page = ID / MAX_PAGE_SIZE;
    int offset = ID % MAX_PAGE_SIZE;

    // Cấp phát/Kiểm tra trạng an toàn
    if (sparse[page].empty()) sparse[page].assign(MAX_PAGE_SIZE, -1);

    slots[page]--; // Giảm số lượng ô trống trong trang hiện tại
    sparse[page][offset] = dense.size(); // Trỏ vị trí mảng thưa tới cuối mảng đặc
    dense.push_back(component); // Nạp dữ liệu vào flat array
    indexs.push_back(ID); // Lưu lại ánh xạ ngược
}

template <typename T>
inline T &SparseSet<T>::get(int ID)
{
    int page = ID / MAX_PAGE_SIZE;
    int offset = ID % MAX_PAGE_SIZE;
    return dense[sparse[page][offset]]; // Ánh xạ 2 bước đạt  $O(1)$ 
}
```

d) Cài đặt kỹ thuật Hủy phần tử dịch chuyển Swap-and-Pop

Thách thức lớn nhất của mảng đặc liên tục là khi xóa một phần tử ở giữa mảng, việc dịch chuyển các phần tử phía sau sẽ tốn chi phí $O(N)$. Hệ thống giải quyết triệt để

bằng thuật toán Swap-and-Pop phối hợp dịch chuyển rỗng (std::move): Lấy phần tử cuối cùng của mảng ghi đè vào vị trí phần tử cần xóa, cập nhật lại các chỉ số ánh xạ tương ứng, rồi thực hiện pop_back() phần tử cuối cùng với chi phí $O(1)$.

```
template <typename T>
inline void SparseSet<T>::remove(int ID)
{
    int ID_Last = indexs.back(); // Lấy ID của thực thể nằm cuối mảng đặc
    int page = ID / MAX_PAGE_SIZE;
    int offset = ID % MAX_PAGE_SIZE;

    // Nếu phần tử cần xóa không phải phần tử cuối cùng, tiến hành hoán đổi vị trí
    if (ID != ID_Last)
    {
        int Pos_Remove = sparse[page][offset]; // Vị trí vật lý cần xóa trong mảng
        // Điều hướng mảng thưa của thực thể cuối cùng về vị trí trống mới hoán đổi
        sparse[ID_Last / MAX_PAGE_SIZE][ID_Last % MAX_PAGE_SIZE] =
        Pos_Remove;
        indexs[Pos_Remove] = ID_Last;

        // Dịch chuyển vùng nhớ của phần tử cuối về vị trí cần xóa với chi phí tối thiểu
        dense[Pos_Remove] = std::move(dense.back());
    }

    // Đánh dấu trống ô nhớ cũ, giải phóng bộ nhớ cuối danh sách
    sparse[page][offset] = -1;
    dense.pop_back();
    indexs.pop_back();

    // Quản lý dọn dẹp trang trống rỗng
    slots[page]++;
    if (slots[page] == MAX_PAGE_SIZE)
        sparse[page].clear(); // Giải phóng hoàn toàn vector trang nếu không còn entity
        nào sử dụng
}
```

4.2.3 SystemManager

a) Kiến trúc lưu trữ tĩnh dựa trên Variadic Templates và Tuple

```
#pragma once
#include "Systems.hpp"
#include <tuple>

template <typename... Systems>
class SystemManager
{
    std::tuple<Systems...> m_systems; // Đóng gói phẳng toàn bộ các System tại thời
    điểm biên dịch

public:
    template <typename T>
    explicit SystemManager(T *scene);

    template <typename T>
    T &get(); // Truy xuất System theo kiểu dữ liệu (Type-safe)

    void init();
    void update();
    void exit();
};
```

b) Hiện thực hóa giải thuật duyệt gói tham số Fold Expressions

```
template <typename... Systems>
template <typename T>
explicit SystemManager<Systems...>::SystemManager(T *scene)
{
    // Sử dụng lambda expression kết hợp Fold Expression để gán con trỏ scene đồng
    loạt
    std::apply([&](auto &...system)
        { (... , (system.scene = scene)); }, m_systems);
}

template <typename T>
T &SystemManager<Systems...>::get()
```

```
{  
    // Truy xuất trực tiếp thực thể System trong bộ nhớ tĩnh qua kiểu dữ liệu T với chi  
    phí O(1)  
    return std::get<T>(m_systems);  
}
```

4.3 Cài đặt Spatial Partitioning

4.3.1 Uniform Grid

a) Định nghĩa cấu trúc ô dữ liệu không gian (Chunk)

Mỗi ô lưới (Chunk) đóng vai trò là một vùng chứa cục bộ. Ngoài nhiệm vụ lưu trữ trạng thái vật lý của địa hình, Chunk quản lý một danh sách động các Entity ID đang nằm trong phạm vi không gian hình học mà nó bao phủ:

```
#pragma once  
#include <vector>  
  
class Chunk  
{  
public:  
    bool block = false;           // Đánh dấu ô lưới bị chặn (Vật cản/Địa hình không  
    thể đi qua)  
    std::vector<int> vec_id_entity; // Danh sách phẳng chứa ID của các thực thể  
    đang đứng trong ô  
  
    // Hủy đăng ký thực thể khỏi ô lưới khi di chuyển ra ngoài  
    void remove(int entityID)  
    {  
        for (int i = 0; i < (int)vec_id_entity.size(); i++)  
        {  
            if (vec_id_entity[i] == entityID)  
            {  
                vec_id_entity.erase(vec_id_entity.begin() + i); // Xóa phần tử khỏi vector  
                break;  
            }  
        }  
    }  
  
    // Đăng ký thực thể vào ô lưới khi di chuyển vào trong (Bảo đảm không trùng lặp)
```



```

void insert(int entityID)
{
    for (int i = 0; i < (int)vec_id_entity.size(); i++)
    {
        if (vec_id_entity[i] == entityID) return; // Tránh hiện tượng nhân bản ID
    }
    vec_id_entity.push_back(entityID);
}

std::vector<int> getEntityInCell() { return vec_id_entity; }
};

```

b) Cài đặt cấu trúc quản lý bản đồ dạng mảng hai chiều (Map)

```

class Map
{
    int m_height = -1; // Chiều cao bản đồ (Tính theo số lượng hàng ô lưới)
    int m_width = -1; // Chiều rộng bản đồ (Tính theo số lượng cột ô lưới)
    std::vector<std::vector<Chunk>> vec_id_entity_in_chunk; // Ma trận quản lý
    không gian

public:
    // Các hàm quản lý trạng thái vật cản (Lock/Unlock) của ô lưới
    bool isLock(int row, int col) { return vec_id_entity_in_chunk[row][col].block; }
    void Lock(int row, int col) { vec_id_entity_in_chunk[row][col].block = true; }
    void UnLock(int row, int col) { vec_id_entity_in_chunk[row][col].block = false; }
}

// Khởi tạo ma trận không gian và cấp phát bộ nhớ dynamic phẳng cho toàn bộ
lưới
void create(int height, int width)
{
    m_height = height;
    m_width = width;
    // Gán cấu hình kích thước và lấp đầy ma trận bằng các thực thể Chunk mặc
    định
    vec_id_entity_in_chunk.assign(height, std::vector<Chunk>(width, Chunk()));
}

```

```

// Thêm thực thể vào tọa độ lưới (row, col) sau khi kiểm tra điều kiện biên an toàn
void insert(int row, int col, int entityID)
{
    if (!isValid(row, col)) return;
    get(row, col).insert(entityID);
}

// Xóa thực thể khỏi tọa độ lưới khi thực thể dịch chuyển sang ô khác hoặc bị hủy
void remove(int row, int col, int entityID)
{
    if (!isValid(row, col)) return;
    get(row, col).remove(entityID);
}

// Truy xuất ô dữ liệu Chunk tại vị trí chỉ định
Chunk &get(int row, int col) { return vec_id_entity_in_chunk[row][col]; }

// Giải thuật kiểm tra an toàn biên, ngăn chặn lỗi tràn mảng (Out of Bounds)
bool isValid(int row, int col)
{
    return row >= 0 && row < m_height && col >= 0 && col < m_width;
}

int width() const { return m_width; }
int height() const { return m_height; }
};

```

4.3.2 Collision Detection System

Xử lý va chạm thực thể theo ô lưới tĩnh (Cell Collision & Trigger Event)

```

void SystemScenePlay::SystemCollision::update()
{
    for (int row = 0; row < scene->map.height(); row++)
    {
        for (int col = 0; col < scene->map.width(); col++)
        {
            auto cell_id = scene->map.get(row, col).getEntityInCell();
            // 1. Xử lý va chạm với ô địa hình bị chặn (Vật cản tĩnh)

```

```

        if (scene->map.isLock(row, col)) {
            for (int id : cell_id) {
                if (scene->entityManager.pool().hasComponent<CVelocity>(id)) {
                    auto &vel = scene-
>entityManager.pool().getComponent<CVelocity>(id);
                    vel.vel.x *= -1; vel.vel.y *= -1;
                }
            }
            continue;
        }
        // 2. Kiểm tra va chạm vòng tròn (Circle Collision) giữa các cặp Entity trong
        cùng ô
        for (int id_1 : cell_id) {
            for (int id_2 : cell_id) {
                if (id_1 < id_2) { // Đảm bảo không kiểm tra trùng lặp
                    auto &pos_1 = scene-
>entityManager.pool().getComponent<CPosition>(id_1);
                    auto &pos_2 = scene-
>entityManager.pool().getComponent<CPosition>(id_2);
                    auto &shape_1 = scene-
>entityManager.pool().getComponent<CShape>(id_1);
                    auto &shape_2 = scene-
>entityManager.pool().getComponent<CShape>(id_2);

                    if (pos_1.pos.dist(pos_2.pos) < shape_1.radius + shape_2.radius) {
                        // Tính toán vector dịch chuyển sửa lỗi chồng lấp hình học
                        sf::Vector2f v12 = pos_2.pos - pos_1.pos;
                        v12 = v12.normalized() * (shape_1.radius + shape_2.radius -
v12.length());

                        pos_1.pos -= v12; // Đẩy lùi thực thể 1

                        // Hủy thực thể 2 và thông báo cho hệ thống tính điểm
                        scene->entityManager.remove(id_2);
                        MessageCollision message{id_1, id_2};
                        scene->eventManager.notify(&message);
                    }
                }
            }
        }
    }
}

```

```

    }
  }
}

```

4.4 Cài đặt Finite State Machine

Hệ thống máy trạng thái hữu hạn (Finite State Machine - FSM) đóng vai trò điều khiển trí tuệ nhân tạo (AI) hành vi của các thực thể (Unit / Agent) trong phân cảnh chiến đấu (SceneCombat). Kiến trúc FSM này được thiết kế theo hướng tiếp cận Event-Driven (Dựa trên sự kiện) kết hợp với mô hình Data-Driven (Hướng dữ liệu) của hệ thống ECS, giúp tối ưu hóa hiệu năng, tách biệt hoàn toàn logic trạng thái khỏi dữ liệu thô của thực thể.

4.4.1 StateMachine template

Để quản lý việc chuyển đổi trạng thái một cách linh hoạt mà không làm phá vỡ kiến trúc thuần dữ liệu của ECS, hệ thống định nghĩa giao diện trừu tượng IState và các lớp bao bọc dữ liệu chuyển cảnh thông qua mẫu thiết kế ComponentBundle.

// Khai báo các kiểu trạng thái và sự kiện cốt lõi của Agent

```
enum class UnitStateKind { Search, Combat, Retreat };
```

```
enum class UnitEventKind { EnemySpotted, TargetLost, HealthLow, Safe };
```

// Giao diện trừu tượng cho một Trạng thái hệ thống

```

struct IState
{
    bool pending = false; // Cờ đánh dấu trạng thái đang đợi xử lý đồng bộ
    [[nodiscard]] virtual UnitStateKind kind() const = 0;

    // Hàm xử lý sự kiện: Trả về một "Gói Component mới" (Bundle) nếu có sự
    // chuyển cảnh
    virtual IComponentBundle* onEvent(SceneCombat* scene, int entityId,
                                     UnitEventKind event, int targetId) const = 0;
    virtual ~IState() = default;
};

```

Ý nghĩa giải thuật: Thay vì trực tiếp thay đổi dữ liệu của Entity bên trong State (gây vi phạm nguyên tắc đóng gói của ECS), hàm onEvent sẽ tính toán và trả về một con trỏ IComponentBundle. Gói Bundle này chứa danh sách các Component cần được thêm, sửa, hoặc xóa khỏi Entity để phục vụ cho trạng thái mới.

4.4.2 Định nghĩa các State cụ thể

Hệ thống cài đặt 3 trạng thái độc lập bao gồm: Tìm kiếm (SearchState), Chiến đấu (CombatState), và Rút lui (RetreatState). Logic chuyển đổi giữa các trạng thái được biểu diễn thông qua sơ đồ cây quyết định dưới đây

a) Trạng thái Tìm kiếm (SearchState)

Mặc định khi sinh ra, Agent sẽ ở trạng thái quét tìm mục tiêu. Khi nhận được sự kiện nhìn thấy kẻ địch (EnemySpotted), hệ thống sẽ tạo ra gói Component chiến đấu tương ứng với mục tiêu đó.

```
UnitStateKind SearchState::kind() const { return UnitStateKind::Search; }

IComponentBundle* SearchState::onEvent(SceneCombat* scene, int entityId,
UnitEventKind event, int targetId) const
{
    if (!scene || pending) return nullptr;

    const bool isAlly = scene->mAdmin.mPool.hasComponent<CAly>(entityId);
    switch (event)
    {
        case UnitEventKind::EnemySpotted:
            if (targetId == -1) return nullptr;
            // Trả về gói Component để chuyển sang trạng thái Tấn công mục tiêu
            return SystemSceneCombat::detail::createCombatBundle(scene, entityId,
isAlly, targetId);
        default:
            return nullptr;
    }
}
```

b) Trạng thái Chiến đấu (CombatState)

Trong trạng thái này, Agent liên tục bám đuổi và tấn công mục tiêu. Hai kịch bản có thể xảy ra: mất dấu kẻ địch (TargetLost) hoặc lượng máu xuống thấp dưới ngưỡng an toàn (HealthLow).

```

UnitStateKind CombatState::kind() const { return UnitStateKind::Combat; }

IComponentBundle* CombatState::onEvent(SceneCombat* scene, int entityId,
UnitEventKind event, int targetId) const
{
    (void)targetId;
    if (!scene || pending) return nullptr;

    const bool isAlly = scene->mAdmin.mPool.hasComponent<CAly>(entityId);
    switch (event)
    {
        case UnitEventKind::TargetLost:
            return SystemSceneCombat::detail::createSearchBundle(scene, entityId,
isAlly);
        case UnitEventKind::HealthLow:
            return SystemSceneCombat::detail::createRetreatBundle(scene, entityId,
isAlly);
        default:
            return nullptr;
    }
}

```

c) Trạng thái Rút lui (RetreatState)

Khi lượng máu nguy hiểm, Agent kích hoạt cơ chế Steering Behavior bỏ chạy ngược hướng kẻ địch. Trạng thái này chỉ kết thúc khi nhận được sự kiện đã an toàn (Safe).

```

UnitStateKind RetreatState::kind() const { return UnitStateKind::Retreat; }

IComponentBundle* RetreatState::onEvent(SceneCombat* scene, int entityId,
UnitEventKind event, int targetId) const
{
    (void)targetId;
    if (!scene || pending) return nullptr;

    const bool isAlly = scene->mAdmin.mPool.hasComponent<CAly>(entityId);
    switch (event)
    {
        case UnitEventKind::Safe:

```

```

        return SystemSceneCombat::detail::createSearchBundle(scene, entityId,
isAlly);
        default:
            return nullptr;
    }
}

```

4.4.3 Tích hợp FSM với ECS

Để tích hợp cơ chế hướng đối tượng của State Pattern vào hệ thống hướng dữ liệu ECS, ta định nghĩa một Component tên là CStateMachine lưu trữ con trỏ trạng thái hiện tại, và một System chuyên trách có nhiệm vụ thực thi vòng lặp (Update) và giải phóng bộ nhớ đệm.

```

// Component lưu trữ dữ liệu trạng thái của một Thực thể trong Pool
struct CStateMachine {
    std::unique_ptr<IState> currentState;
    UnitStateKind stateKind;
};

```

Khi nhận được sự kiện hay đủ điều kiện kích hoạt sự kiện System State Machine sẽ tạo 1 sự kiện chuyển trạng thái cho entity đó. Có 1 số trạng thái cần update liên tục thì sẽ được cập nhật trong System StateMachine

4.5 Cài đặt Network Layer

Tầng mạng (Network Layer) chịu trách nhiệm đồng bộ hóa trạng thái trò chơi giữa Client và Server trong môi trường mạng có độ trễ. Trong kiến trúc Game Engine đang phát triển, tầng mạng được cấu trúc hóa thông qua lớp SceneNet, tích hợp đồng thời hai thành phần logic độc lập: SNet::ClientGame (Quản lý dự đoán phía Client) và SNet::ServerGame (Cơ quan tối cao quyết định trạng thái chính xác). Sự tương tác dữ liệu được đóng gói gọn gàng trong cấu trúc gói tin SNet::Package:

```

struct Package {
    size_t type;           // Loại gói tin
    size_t frame;          // Số thứ tự khung hình (Frame Index)
    Vec2i pos;             // Tọa độ vật lý của thực thể
    Time time;             // Trạng thái mốc thời gian của gói
    Input input;           // Dữ liệu đầu vào (left, right)
    State state;           // Trạng thái mô phỏng (match, frame, value)
}

```

```
};
```

4.5.1 Giả lập độ trễ mạng

Trong môi trường kiểm thử cục bộ, độ trễ mạng thực tế (Ping / RTT) được giả lập bằng cách sử dụng các hằng số thời gian cố định và cơ chế tích lũy thời gian biến thiên (lag).

```
constexpr double tickRate    = 1000.f / 5.f; // Chu kỳ tick mạng (200ms mỗi tick)
constexpr double lag_client  = tickRate * 8.f; // Độ trễ gửi từ Client lên Server
(1600ms)
constexpr double lag_server  = tickRate * 6.f; // Độ trễ gửi từ Server về Client
(1200ms)
constexpr double gap_package = 400.f;        // Khoảng cách đồ họa biểu diễn gói
tin
```

Cơ chế dịch chuyển thời gian và truyền gói tin trì hoãn:

Tại mỗi hàm update(), biến cur lấy mốc thời gian thực của hệ thống, sau đó cập nhật đại lượng time.lag cho cả Client, Server và các gói tin đang bay trên đường truyền (sending). Quá trình truyền phát gói tin chỉ thực sự hoàn tất khi time.lag của gói tin vượt qua ngưỡng trễ quy định (lag_client hoặc lag_server):

// Minh họa cơ chế đẩy gói tin từ hàng đợi 'sending' sang 'received' dựa trên độ trễ

```
while (m_client.sending.size() && m_client.sending.front().time.lag > lag_client)
{
    if (m_client.sending.front().frame > m_server.frame)
        m_server.received.push_back(m_client.sending.front());
    m_client.sending.pop_front();
}

while (m_server.sending.size() && m_server.sending.front().time.lag > lag_server)
{
    m_client.received.push_back(m_server.sending.front());
    m_server.sending.pop_front();
}
```

4.5.2 Input Buffering

Để ngăn chặn hiện tượng giật đứng hình ảnh khi các gói tin từ Client đến Server bị dồn cục hoặc đứt quãng, hệ thống triển khai giải thuật Input Buffering tại phía Server thông qua hàng đợi `m_server.received`, kết hợp với kỹ thuật Điều chỉnh tốc độ thực thi động (Dynamic Speed Throttling) ở phía Client.

Hệ thống sẽ liên tục giám sát số lượng gói tin đầu vào hiện có trong hàng đợi đệm của Server (`buffer = m_server.received.size()`) để điều phối lại tốc độ cập nhật dòng thời gian (`m_client.speed`) của Client:

```
size_t buffer = m_server.received.size();
```

```
// Giải thuật điều khiển tốc độ mô phỏng mạng của Client dựa trên hàng đợi của Server
```

```
m_client.speed = (buffer == 0) ? 1.2f : (buffer >= 3) ? 0.4f : 1.0f;
```

Phân tích giải thuật:

- Trường hợp `buffer == 0` (Thiếu hụt dữ liệu - Starvation): Server không nhận được lệnh từ Client do lag. Client sẽ tự động tăng tốc độ xử lý lên 120% (`speed = 1.2f`) để nhanh chóng đẩy thêm gói tin nạp đầy bộ đệm của Server.
- Trường hợp `buffer >= 3` (Tràn bộ đệm - Overflow): Các gói tin Client dồn ứ tại Server quá nhiều. Client sẽ tự động hãm phanh tốc độ xuống chỉ còn 40% (`speed = 0.4f`) để chờ Server tiêu thụ hết dữ liệu cũ, tránh hiện tượng nghẽn mạch.
- Trường hợp lý tưởng ($0 < \text{buffer} < 3$): Client duy trì tốc độ tiêu chuẩn 1.0f.

4.5.3 Client-side Prediction & Reconciliation

Đây là kỹ thuật cốt lõi giúp triệt tiêu cảm giác trễ cho người chơi. Client không đợi phản hồi từ Server mà tự động áp đặt Input để tính toán trước trạng thái (Client-side Prediction), đồng thời lưu các trạng thái dự đoán này vào hàng đợi `state_pendings`. Khi Server gửi trạng thái chuẩn về, Client thực hiện so khớp và sửa sai nếu phát hiện lệch dữ liệu (Reconciliation).

a) Logic Dự đoán phía Client (Client-side Prediction)

Mỗi khi Client chạy một Tick mạng (`m_client.m_time.lag - tickRate > 0`), nó tăng chỉ số Frame, tính toán vị trí giả lập mới (`pkg.pos.x = gap_update * m_client.frame`) và đẩy một cấu trúc dự đoán `SNet::State` vào mảng `state_pendings`. Nhằm kiểm thử tính bền vững của giải thuật, mã nguồn cố tình gài một tỷ lệ lỗi ngẫu nhiên (`rand() % 100`) để mô phỏng sự sai lệch vị trí:

```

while (m_client.m_time.lag - tickRate > 0)
{
    m_client.frame++;
    m_client.m_time.lag -= tickRate;

    // ... Khởi tạo gói tin gửi đi ...
    m_client.sending.push_back(pkg);

    if (m_client.state_pendings.size())
    {
        SNet::State state;
        state.match = 1; // Mặc định giả định là khớp (Đúng đắn)
        state.frame = m_client.frame;

        // Cố tình tạo sai số ngẫu nhiên để kiểm tra bộ lọc Reconciliation
        state.value = m_client.state_pendings.back().value + 1 +
            ((rand() % 100) == 50 || (rand() % 100) == 40 || (rand() % 100) == 45);

        m_client.state_pendings.push_back(state);
    }
}

```

b) Giải thuật Đồng bộ và Sửa sai (Server Reconciliation)

Khi nhận được gói tin phản hồi từ Server (m_client.received), Client tiến hành trích xuất Frame lịch sử. Nó loại bỏ toàn bộ các trạng thái cũ hơn Frame này trong state_pendings. Tiếp theo, nó lấy trạng thái dự đoán ứng với Frame đó ra so khớp với giá trị có thẩm quyền của Server (pkg.state.value).

Nếu xảy ra hiện tượng lệch trạng thái (!predicted_state.match), Client lập tức kích hoạt vòng lặp Replay. Vòng lặp này lấy giá trị chuẩn của Server làm gốc, sau đó tính toán cuốn chiếu tuần tự lại toàn bộ các Frame kế tiếp đang nằm chờ trong hàng đợi để cập nhật lại chuỗi vị trí chính xác:

```

while (m_client.received.size())
{
    auto pkg = m_client.received.front();
    m_client.received.pop_front();
    size_t frame = pkg.state.frame;

```

```

// Loại bỏ các frame cũ hơn mốc thời gian của Server
while (m_client.state_pendings.size() && m_client.state_pendings.front().frame
< frame)
    m_client.state_pendings.pop_front();

// Thực hiện kiểm tra so khớp trạng thái (Dự đoán vs Thực tế Server)
auto &predicted_state = m_client.state_pendings.front();
predicted_state.match = (predicted_state.value == pkg.state.value);

if (!predicted_state.match) // PHÁT HIỆN SAI LỆCH (DESYNC)
{
    // 1. Áp đặt lại giá trị chuẩn xác từ Server cho frame lệch sử này
    predicted_state.value = pkg.state.value;

    // 2. RECONCILIATION REPLAY: Chạy lại logic toán học cho toàn bộ chuỗi
    các frame tương lai
    auto it = m_client.state_pendings.begin();
    auto next_it = ++m_client.state_pendings.begin();
    while (next_it != m_client.state_pendings.end())
    {
        next_it->match = 0;           // Đánh dấu lại các khối đồ họa trên màn hình là
        bị lỗi (Red Box)
        next_it->value = it->value + 1; // Tính toán lại giá trị đúng đắn dựa trên
        frame gốc liền trước
        it = next_it;
        ++next_it;
    }
}
}

```

Hiệu quả đồ họa trực quan trên ImGui/SFML:

Nhờ giải thuật này, trên giao diện kết xuất đồ họa (sRender()), các hộp trạng thái `state_pendings` dự đoán đúng sẽ được vẽ với màu Tím (`sf::Color::Magenta`), còn các trạng thái bị sai lệch ngay lập tức bị phát hiện, bôi Đỏ (`sf::Color::Red`), và hệ thống sẽ tự động ép dòng dữ liệu sửa sai về đúng quỹ đạo chuyển động mượt mà của Server mà người chơi không hề nhận ra sự gián đoạn.

4.6 Cài đặt Profiler

Để tối ưu hóa hiệu năng, phát hiện các điểm nghẽn (Bottlenecks) trong hệ thống ECS và Network Layer, đồ án cài đặt một bộ công cụ đo kiểm cấu trúc tự động (Instrumentation Profiler). Hệ thống tận dụng cơ chế RAII dựa trên vòng đời của biến cục bộ thông qua lớp ProfileTimer và cấu trúc quản lý tập trung Profiler (Singleton). Dữ liệu thu được cấu trúc hóa theo định dạng JSON chuẩn giúp trực quan hóa dòng thời gian trực tiếp trên giao diện Chrome Tracing (chrome://tracing hoặc Perfetto UI).

```
struct ProfileResult {
    std::string name = "Default"; // Tên hàm hoặc phân đoạn mã được đo
    long long start = 0;           // Mốc thời gian bắt đầu (Microseconds)
    long long end = 0;             // Mốc thời gian kết thúc (Microseconds)
    size_t threadID = 0;           // Định danh luồng (Thread ID) thực thi
};

void ProfileTimer::start() {
    static long long lastStartTime = 0;
    m_startTimepoint = std::chrono::high_resolution_clock::now();
    m_result.start =
std::chrono::time_point_cast<std::chrono::microseconds>(m_startTimepoint).time_s
ince_epoch().count();

    // Sửa lỗi trùng mốc thời gian gây lỗi hiển thị đồ họa
    m_result.start += (m_result.start == lastStartTime) ? 1 : 0;
    lastStartTime = m_result.start;
    m_stopped = false;
}

void writeProfile(const ProfileResult &result) {
    std::lock_guard<std::mutex> lock(m_lock); // Đảm bảo an toàn đa luồng (Thread-
safe)

    if (m_profileCount++ > 0) { m_outputStream << ","; }

    std::string name = result.name;
    std::replace(name.begin(), name.end(), '"', "\\"); // Chuẩn hóa chuỗi

    m_outputStream << "\n{";
    m_outputStream << "\"cat\":\"function\",";
```

```

    m_outputStream << "\"dur\":" << (result.end - result.start) << ','; // Thời gian thực
    thi
    m_outputStream << "\"name\":" << name << "\",";
    m_outputStream << "\"ph\":" << "X\","; // Cấu hình Complete Event (Thời gian có
    mốc đầu/cuối)
    m_outputStream << "\"pid\":" << "0,";
    m_outputStream << "\"tid\":" << result.threadID << ","; // Phân nhóm theo
    Thread trên UI
    m_outputStream << "\"ts\":" << result.start; // Thời điểm bắt đầu
    m_outputStream << "}";
}

```

CHƯƠNG 5 : THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 Đánh giá hiệu năng kiến trúc ECS

5.1.1 Profiling

Kịch bản thiết lập:

- Mô phỏng: Thực hiện di chuyển, cập nhật vòng đời (lifespan), kiểm tra va chạm biên và render hàng loạt các thực thể hình học.
- Kiến trúc OOP: Sử dụng `std::vector<std::shared_ptr<OOP::Entity>>`.
- Kiến trúc ECS: Sử dụng MemoryPool

5.1.2 Kết quả : bảng + biểu đồ so sánh

TH1 : 30000 thực thể

10985 items selected.		Slices (10985)			
Name ▾		Wall Duration ▾	Self time ▾	Average Wall Duration ▾	Occurrences ▾
update		10,685.546 ms	84.366 ms	4.864 ms	2197
SceneOOP::sSpawnEntity		1,675.119 ms	1,675.119 ms	0.762 ms	2197
SceneOOP::sLifeSpan		3,445.882 ms	3,445.883 ms	1.568 ms	2197
SceneOOP::sMovement		4,064.284 ms	4,064.284 ms	1.850 ms	2197
SceneOOP::sCollision		1,415.894 ms	1,415.894 ms	0.644 ms	2197
Totals		21,286.725 ms	10,685.546 ms	1.938 ms	10985
Selection start		0.000 ms			
Selection extent		35,147.455 ms			

14208 items selected.		Slices (14208)			
Name ▾		Wall Duration ▾	Self time ▾	Average Wall Duration ▾	Occurrences ▾
update		5,799.161 ms	105.445 ms	2.449 ms	2368
SceneECS::m_admin.update		2,377.203 ms	2,377.203 ms	1.004 ms	2368
SceneECS::sSpawnEnemy		1,604.465 ms	1,604.465 ms	0.678 ms	2368
SceneECS::sLifeSpan		689.076 ms	689.076 ms	0.291 ms	2368
SceneECS::sMovement		730.995 ms	730.995 ms	0.309 ms	2368
SceneECS::sCollision		291.977 ms	291.977 ms	0.123 ms	2368
Totals		11,492.877 ms	5,799.161 ms	0.809 ms	14208
Selection start					0.000 ms
Selection extent					37,871.131 ms

TH2 : 60000 thực thể

13010 items selected.		Slices (13010)			
Name ▾		Wall Duration ▾	Self time ▾	Average Wall Duration ▾	Occurrences ▾
update		20,765.704 ms	112.274 ms	7.981 ms	2602
SceneOOP::sSpawnEntity		4,509.935 ms	4,509.935 ms	1.733 ms	2602
SceneOOP::sLifeSpan		5,942.952 ms	5,942.952 ms	2.284 ms	2602
SceneOOP::sMovement		7,021.397 ms	7,021.397 ms	2.698 ms	2602
SceneOOP::sCollision		3,179.146 ms	3,179.146 ms	1.222 ms	2602
Totals		41,419.134 ms	20,765.704 ms	3.184 ms	13010
Selection start					0.000 ms
Selection extent					41,637.995 ms

17838 items selected.		Slices (17838)			
Name ▾		Wall Duration ▾	Self time ▾	Average Wall Duration ▾	Occurrences ▾
update		11,847.907 ms	146.551 ms	3.985 ms	2973
SceneECS::m_admin.update		5,322.123 ms	5,322.123 ms	1.790 ms	2973
SceneECS::sSpawnEnemy		2,913.896 ms	2,913.896 ms	0.980 ms	2973
SceneECS::sLifeSpan		1,312.812 ms	1,312.812 ms	0.442 ms	2973
SceneECS::sMovement		1,617.065 ms	1,617.065 ms	0.544 ms	2973
SceneECS::sCollision		535.460 ms	535.460 ms	0.180 ms	2973
Totals		23,549.263 ms	11,847.907 ms	1.320 ms	17838
Selection start					0.000 ms
Selection extent					47,550.159 ms

5.1.3 Phân tích và nhận xét

- + Về mặt tổng thể thì kiến trúc ECS do CPU prefetch dữ liệu tốt hơn nên cho kết quả nhanh hơn khoảng 2.5 lần so với cách làm truyền thống
- + Một số hàm trong ECS nhanh hơn khoảng 5 đến 6 lần như hàm sLifespan và hàm sMovement và hàm sCollision
 - + Lý do các hàm đó nhanh hơn là do các hàm đó trong kiến trúc trên dữ liệu được duyệt tuyến tính nên hiện tượng cache hit nhiều dẫn đến kết quả nhanh hơn nhiều

5.2 Đánh giá Sparse Set

5.2.1 So sánh tốc độ add/remove/iterate với std::unordered_map

- + Add: Sparse Set đẩy trực tiếp vào cuối mảng phẳng, không tốn chi phí tính hàm băm như std::unordered_map, do đó tốc độ thêm nhanh hơn đáng kể khi số lượng phần tử lớn.
- + Remove: Sparse Set dùng Swap-and-Pop đạt $O(1)$ mà không làm phân mảnh bộ nhớ. std::unordered_map phải giải phóng node động, làm tăng phân mảnh Heap theo thời gian.
- + Iterate: Sparse Set duyệt tuyến tính trên mảng phẳng liên tiếp, CPU có thể prefetch dữ liệu tối đa. std::unordered_map lưu dạng node rải rác trên Heap, gây Cache Miss liên tục khi duyệt, làm tốc độ chậm hơn rõ rệt.

5.2.2 Kết quả benchmark

Output:

```
===== ADD =====
SparseSet Add : 67.3489 ms
unordered_map Add : 152.612 ms

===== FULL TEST =====

===== GET =====
SparseSet Get : 27.9077 ms
unordered_map Get : 51.3979 ms

===== ITERATE =====
SparseSet Iterate : 6.41036 ms
unordered_map Iterate : 8.21207 ms

===== REMOVE =====
SparseSet Remove : 114.34 ms
unordered_map Remove : 202.682 ms
```

5.3 Đánh giá FSM

5.3.1 Ưu điểm

- Trực quan, dễ hiểu: Dễ dàng mô hình hóa hệ thống qua sơ đồ trạng thái, giúp các bên liên quan dễ nắm bắt.
- Tính xác định cao (Deterministic): Hệ thống hoạt động nhất quán, dự đoán trước được kết quả dựa trên trạng thái và đầu vào.
- Dễ kiểm thử và gỡ lỗi: Số lượng trạng thái hữu hạn giúp việc tạo kịch bản kiểm thử (test cases) và tìm vết lỗi nhanh chóng.
- Hiệu suất cao: Triển khai bằng mã nguồn đơn giản (switch-case) hoặc mạch logic nên tốc độ xử lý cực nhanh và tốn ít bộ nhớ.

5.3.2 Nhược điểm

- Bùng nổ trạng thái (State Explosion): Khi hệ thống phức tạp, số lượng trạng thái tăng lũy tiến, khiến sơ đồ bị rối và khó quản lý.
- Kém linh hoạt với hệ thống lớn: Khó xử lý các tác vụ song song hoặc các logic có độ phức tạp cao.
- Không có bộ nhớ lịch sử: FSM chỉ biết trạng thái hiện tại, không tự lưu trữ lộ trình các trạng thái đã đi qua.
- Khó tái sử dụng: Thiết kế thường mang tính đặc thù cho một bài toán cụ thể, khó chuyển giao sang dự án khác.

5.4 Đánh giá kỹ thuật mạng

5.4.1 Độ trễ cảm nhận : có với không có Client-side Prediction

1. Trường hợp KHÔNG CÓ Client-side Prediction

- Cơ chế hoạt động: Khi người chơi nhấn nút di chuyển, Client chỉ gửi gói tin Input lên Server và đứng yên chờ đợi. Chỉ khi Server xử lý xong, cập nhật tọa độ mới và gửi gói tin trạng thái về, Client mới cập nhật vị trí nhân vật trên màn hình.
- Độ trễ cảm nhận (Perceived Latency): Rất cao (Bằng đúng RTT - Round Trip Time).
 - Người chơi sẽ cảm thấy game bị "khựng" (input lag) rõ rệt.
 - Nếu mạng có Ping cao hoặc mất gói (packet loss), nhân vật sẽ phản hồi cực kỳ chậm chạp, gây ức chế lớn cho trải nghiệm người dùng.

2. Trường hợp CÓ Client-side Prediction

- Cơ chế hoạt động: Ngay khi người chơi nhấn nút, Client lập tức cập nhật vị trí nhân vật trên màn hình cục bộ dựa trên logic vật lý/di chuyển của chính nó, đồng thời gửi Input lên Server. Khi Server gửi kết quả về, Client sẽ đối chiếu. Nếu có sai lệch (do lag, va chạm vật lý phía Server), Client sẽ thực hiện hòa giải trạng thái (Reconciliation) để sửa lại vị trí.
- Độ trễ cảm nhận (Perceived Latency): Gần như bằng 0 (Instant feedback).
 - Người chơi có cảm giác nhân vật phản hồi ngay lập tức, mượt mà và không phụ thuộc vào tốc độ đường truyền (Ping).
 - Che giấu hoàn toàn độ trễ mạng trong điều kiện kết nối ổn định.

5.4.2 Ưu điểm và nhược điểm

Ưu điểm:

- Client-side Prediction loại bỏ cảm giác giật lag, nhân vật phản hồi tức thì dù đường truyền có độ trễ cao.

- Input Buffering giúp Server xử lý dữ liệu ổn định, tránh hiện tượng nhân vật giật cục khi gói tin đến không đều.
- Reconciliation đảm bảo tính chính xác của trạng thái game, Server luôn là cơ quan quyết định cuối cùng, ngăn chặn gian lận từ phía Client.

Nhược điểm:

- Khi sai lệch lớn, người chơi có thể nhận ra hiện tượng nhân vật bị kéo ngược vị trí đột ngột (rubber-banding).
- Input Buffering đánh đổi một khoảng trễ nhỏ cố định, có thể ảnh hưởng trong các thể loại game yêu cầu phản xạ cực cao.
- Mỗi lần Reconciliation xảy ra, Client phải tính lại toàn bộ chuỗi Frame từ điểm sai lệch, gây áp lực lên CPU nếu tần suất sai lệch cao.

CHƯƠNG 6 : KẾT LUẬN

6.1 Kết quả đạt được

Các phần dừng lại ở mức mô phỏng

- + Client-side prediction

Các phần đã ứng dụng vào trong sản phẩm

- + Sử dụng ECS và SparseSet để cải thiện hiệu suất và dễ cấu trúc chương trình
- + Sử dụng State Machine và Lưới điều hướng để điều khiển hành vi nhân vật dễ dàng hơn
- + Sử dụng Uniform Grid để phân vùng không gian giúp tìm kiếm thực thể nhanh chóng hơn

6.2 Hạn chế

- + Do kỹ thuật Client-side Prediction mới chỉ dừng ở mức mô phỏng , trò chơi chưa được kiểm thử toàn diện trên môi trường mạng thực tế
- + Giới hạn của Uniform Grid : Kỹ thuật này gây tốn bộ nhớ hơn khi các thực thể không phân bố đều
- + Nếu có nhiều trạng thái thì máy trạng thái sẽ khó kiểm soát

6.3 Hướng phát triển

- + Hoàn thiện Client-side Prediction để sản phẩm có thể chạy trên môi trường mạng thực tế
- + Tìm hiểu các mô hình khác để điều khiển hành vi AI thay vì chỉ dựa vào máy trạng thái